

A PROJECT REPORT ON
SMART HOME ASSISTANT USING ESP32 AND VOICE COMMANDS



Submitted to
Jawaharlal Nehru Technological University Anantapur, Anantapuramu

In partial fulfilment for the Award of the degree of
BACHELOR OF TECHNOLOGY
In
ELECTRONICS & COMMUNICATION OF ENGINEERING

Submitted by
INJARAPU SIVA KRISHNA (22JN1A04D3)
SHAIK ABRARUL HUK (22JN1A04E0)
SHAIK MOHAMMED THAHIR (22JN1A04E6)
SRIPATHI VAMSI KISHORE (22JN1A04F8)

Under the Esteemed Guidance of
Mr. A. HARI KRISHNA, M.Tech.,
ASSISTANT PROFESSOR

Dept. of ECE

SREE VENKATESWARA COLLEGE OF ENGINEERING

Accredited with NAAC “A”, UGC 2(f) Recognized & ISO 9001::2015 Certified,
Approved by AICTE, New Delhi and Affiliated to JNTUA – Anantapuramu
North Rajupalem, Kodavaluru(V&M), Nellore



2022-2026

A PROJECT REPORT ON
SMART HOME ASSISTANT USING ESP32 AND VOICE COMMANDS



Submitted to

Jawaharlal Nehru Technological University Anantapur, Anantapuramu

In partial fulfilment for the Award of the degree of

BACHELOR OF TECHNOLOGY

In

ELECTRONICS & COMMUNICATION OF ENGINEERING

Submitted by

INJARAPU SIVA KRISHNA (22JN1A04D3)

SHAIK ABRARUL HUK (22JN1A04E0)

SHAIK MOHAMMED THAHIR (22JN1A04E6)

SRIPATHI VAMSI KISHORE (22JN1A04F8)

Under the Esteemed Guidance of

Mr. A. HARI KRISHNA, M.Tech.,

ASSISTANT PROFESSOR

Dept. of ECE

SREE VENKATESWARA COLLEGE OF ENGINEERING

Accredited with NAAC “A”, UGC 2(f) Recognized & ISO 9001::2015 Certified,

Approved by AICTE, New Delhi and Affiliated to JNTUA – Anantapuramu

North Rajupalem, Kodavaluru(V&M), Nellore



2022-2026



NAAC 'A' Grade Accredited Institution

ISO 9001:2015 Certified

SREE VENKATESWARA COLLEGE OF ENGINEERING

(UGC AUTONOMOUS INSTITUTION)

Approved by AICTE, New Delhi & Affiliated to JNTUA, Anantapuram
Northrajapalem (Vi), Kodavaluru (M), S.P.S.R Nellore (Dt)-524316

EAPCET(EAMCET) CODE: SVCN

ECET CODE : SVCN

POLYCET CODE : SVCN



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

CERTIFICATE

This is to certify that the project entitled **SMART HOME ASSISTANT USING ESP32 AND VOICE COMMANDS** submitted by **INJARAPU SIVA KRISHNA (22JN1A04D3), SHAIK ABRARUL HUK (22JN1A04E0), SHAIK MOHAMMED THAHIR (22JN1A04E6), SRIPATHI VAMSI KISHORE (22JN1A04F8)** in partial fulfilment of for the award of degree of **BACHELOR TECHNOLOGY** in **ELECTRONICS AND COMMUNICATION ENGINEERING** by **Jawaharlal Nehru Technological University Anantapur, Anantapuramu** during the academic year 2022-2026. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Signature of the Guide

Mr. A. HARI KRISHNA, M.Tech.,
Assistant Professor

Signature of Head of the Dept

Dr. B. BABU, M. Tech, Ph.D.,
Associate Professor

External VIVA-VOCE conducted on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

With immense respect, we express our sincere gratitude to **Dr. P. GUNA SEKHAR, Chairman, SV Group of Institutions** and **Dr. M. SREENIVASA KUMAR, Principal, Sree Venkateswara College of Engineering** for permitting us to take up our project work and to complete the project successfully.

We take immense pleasure in expressing our sincere thanks and deep sense of gratitude to **Dr.B. BABU, M. Tech, Ph.D., Associate Professor and Head of the Department, Electronics and Communication Engineering**, for extending necessary facilities for the completion of the project.

I sincerely thank to **Mrs. P. Mohana, M.E., Assistant Professor, Department of Electronics and Communication Engineering** for her excellent guidance and continuous support. Her strong technical knowledge helped us overcome challenges and successfully implement our project.

We are highly thankful to **Mr. A. HARI KRISHNA, M.Tech., Assistant Professor, Department of Electronics and Communication Engineering**, for **his** inspiring guidance and for providing the background knowledge to deal with the problem at every phase of our project in a systematic manner.

We would like to express our sincere thanks to all faculty members of the department for their continuous cooperation, which has given us the cogency to build-up adamant aspiration for the completion of our project.

Finally, we thank one and all who directly and indirectly helped us to complete the project successfully.

TEAM MEMBERS

INJARAPU SIVA KRISHNA (22JN1A04D3)

SHAIK ABRARUL HUK (22JN1A04E0)

SHAIK MOHAMMED THAHIR (22JN1A04E6)

SRIPATHI VAMSI KISHORE (22JN1A04F8)

DECLARATION

We, **INJARAPU SIVA KRISHNA (22JN1A04D3)**, **SHAIK ABRARUL HUK (22JN1A04E0)**, **SHAIK MOHAMMED THAHIR (22JN1A04E6)**, **SRIPATHI VAMSI KISHORE (22JN1A04F8)** do hereby declare that the project entitled “SMART HOME ASSISTANT USING ESP-32 AND VOICE COMMANDS” was carried out by our batch under the guidance of **Mr. A. HARI KRISHNA Assistant Professor, ECE**. This Project work is submitted to Jawaharlal Nehru Technological University Anantapur, Anantapuramu, in fulfilment of the requirement for the award of a **BACHELOR OF TECHNOLOGY in ELECTRONICS AND COMMUNICATION ENGINEERING** during the academic year 2025- 2026.

TEAM MEMBERS

INJARAPU SIVA KRISHNA (22JN1A04D3)

SHAIK ABRARUL HUK (22JN1A04E0)

SHAIK MOHAMMED THAHIR (22JN1A04E6)

SRIPATHI VAMSI KISHORE (22JN1A04F8)

ABSTRACT

The Smart Home Assistant using ESP-32 and voice commands is an innovative system designed to automate and control household appliances through voice interaction. This project utilizes the ESP-32 microcontroller due to its built-in Wi-Fi and Bluetooth capabilities, enabling seamless communication between the user and connected devices. The system integrates voice recognition technology, allowing users to issue commands through a smartphone or voice-enabled interface.

The primary objective of this project is to enhance convenience, energy efficiency, and accessibility in modern homes. Users can control devices such as lights, fans, and other electrical appliances remotely or through simple voice instructions. The ESP-32 processes the received commands and triggers the corresponding actions via relay modules.

Additionally, the system can be expanded to include features such as scheduling, real-time monitoring, and integration with IOT platforms. This project demonstrates a cost-effective and scalable solution for home automation, making it suitable for smart living environments. Overall, the Smart Home Assistant improves user comfort and contributes to the development of intelligent and energy-efficient homes.

The Smart Home Assistant using ESP-32 and voice commands is a modern home automation solution that integrates embedded systems with Internet of Things (IOT) technology. The system leverages the ESP-32 microcontroller for its high processing capability and built-in wireless features. Voice commands are captured through a user interface and processed using speech recognition algorithms.

Upon receiving commands, the ESP-32 communicates with connected hardware components to control home appliances via relay circuits. The system is designed to provide real-time operation, remote accessibility, and efficient energy utilization. Furthermore, it offers scalability for future enhancements such as sensor integration, automation scheduling, and AI-based decision-making.

This project aims to simplify daily tasks, improve energy efficiency, and contribute to the advancement of smart living environments.

With the rapid advancement of IOT technologies, smart home systems are becoming increasingly popular. This project introduces a Smart Home Assistant using ESP-32 that enables voice-based control of household appliances.

Voice commands are interpreted and executed in real-time, providing a seamless user experience. The ESP-32 acts as the central controller, ensuring reliable and fast operation. This solution not only enhances convenience but also promotes energy conservation and modern living standards.

TABLE OF CONTENTS

CHAPTER -1: INTRODUCTION

1.1 INTRODUCTION	2
1.2 PROBLEM STATEMENT	3
1.3 PRESEARCH MOTIVATION	4
1.4 APPLICATIONS	5

CHAPTER – 2: LITERATURE SURVEY

2.1 OVERVIEW – EXISTING HOME AUTOMATION SYSTEM	7
2.2 VOICE-CONTROLLED SYSTEMS	8
2.3 COMPARISON OF TECHNOLOGIES	9
2.4 LIMITATIONS OF EXISTING SYSTEMS	10

CHAPTER – 3: EXISTING SYSTEM

3.1 EXISTING SYSTEM	11
3.2 LIMITATIONS OF EXISTING SYSTEM	12

CHAPTER – 4: PROPOSED SYSTEM

4.1 PROPOSED SYSTEM	15
4.2 METHODOLOGY	16
4.3 WORK FLOW	18
4.4 ADVANTAGES	20

CHAPTER – 5: SYSTEM ARCHITECTURE

5.1 BLOCK DIAGRAM	23
5.2 OVERALL SYSTEM DESIGN	24
5.3 WORKING PRINCIPLE	27
5.4 DATA FLOW DIAGRAM.....	28
5.5 COMPONENT DIAGRAM	29

CHAPTER – 6: HARDWARE COMPONENTS

6.1 ESP-32 MICROCONTROLLER	32
6.2 RELAY MODULE.....	33
6.3 POWER SUPPLY UNIT	37
6.4 ELECTRICAL APPLIANCES	40
6.5 CONNECTING WIRES AND CIRCUIT SETUP	41

CHAPTER – 7: SOFTWARE COMPONENTS

7.1 ARDUINO IDE / PROGRAMMING ENVIRONMENT	44
7.2 EMBEDDED C.....	46
7.3 VOICE RECOGNITION MODULE / APPLICATION	47
7.4 IOT PLATFORM	49
7.5 MOBILE APPLICATION INTEGRATION	50

CHAPTER – 8: IMPLEMENTATION

8.1 CIRCUIT DIAGRAM	53
8.2 HARDWARE SETUP	55
8.3 SOFTWARE DEVELOPMENT	57
8.4 INTEGRATION OF VOICE COMMANDS	60
8.5 TESTING PROCEDURE	61

CHAPTER – 9: WORKING OF THE SYSTEM

9.1 VOICE COMMAND PROCESSING	66
9.2 COMMUNICATION VIA BLUETOOTH	67
9.3 DEVICE CONTROL MECHANISM	74

CHAPTER – 10: RESULTS AND DISCUSSION

10.1 OUTPUT RESULTS	76
---------------------------	----

CHAPTER – 11: CONCLUSION AND FUTURE SCOPE

11.1 CONCLUSION	81
11.2 FUTURE SCOPE.....	82
11.3 REFERENCES	83

LIST OF FIGURES

NUMBER	TITLE	PAGE NUMBER
4.1	System architecture of Smart Home Assistant using ESP-32	16
5.1	Block Diagram of Smart Home Assistant using ESP-32	24
5.2	System Operation Flow	26
5.4	Data Flow Diagram	28
5.5	Component Diagram of Smart Home Automation	30
6.1	ESP-32 Micro Controller	32
6.2	Relay Module	33
6.2.1	Typical Relay	34
6.2.2	SPST Relay	35
6.2.3	DPDT Relay	36
6.3	Power Supply Unit & Rectifier & Filters	39
6.4	Electrical Appliances	40
6.5	Connecting Wires And Circuit Setup	41
7.2	Embedded C Language	46
7.3	Voice Recognition / Application	47
7.4	IOT Platform	49
7.5	Mobile Application Integration (Arduino Bluetooth Control)	50
8.1	Circuit Diagram of Smart Home Assistant using ESP-32	53
8.3	Software Development Tool	58
8.4	Integration of Voice Commands	60
8.5	Testing Procedure of ESP-32	61
9.1	Voice Command Processing	65
9.2	Communication Via Bluetooth	66
9.3	Block Diagram of Device Control Mechanism	73

CHAPTER 1

CHAPTER-1: INTRODUCTION

1.1 Introduction

The Smart Home Assistant using voice commands is an innovative application of the Internet of Things (IoT) that enables users to control household appliances through simple spoken instructions. In today's fast-paced world, automation has become an essential part of modern living, improving comfort, convenience, and efficiency. This project focuses on developing a system where devices such as lights, fans, and other electrical appliances can be controlled remotely and hands-free using voice commands.

At the core of this system is the ESP32, a powerful and cost-effective microcontroller with built-in Wi-Fi and Bluetooth capabilities. The ESP32 acts as the central processing unit that receives commands, processes them, and controls the connected devices accordingly. Its ability to connect directly to the internet makes it highly suitable for IoT-based applications, eliminating the need for additional communication modules.

The system works by capturing voice input from the user through a smartphone or a voice-enabled platform. This input is processed using applications or services such as Blynk or voice assistants like Google Assistant. The interpreted command is then transmitted over a Wi-Fi network to the ESP32. Based on the received instruction, the ESP32 activates or deactivates relays connected to various appliances, thereby controlling their operation.

One of the key motivations behind this project is to provide a user-friendly and accessible automation system, especially for elderly individuals and people with physical disabilities who may find it difficult to operate traditional switches. By integrating voice control, the system eliminates the need for manual interaction, making it more inclusive and efficient.

Additionally, this project contributes to energy conservation by allowing users to monitor and control devices remotely, ensuring that appliances are not left running unnecessarily. Compared to commercially available smart home systems, this design is affordable, customizable, and easy to implement, making it ideal for educational purposes and small-scale applications.

In summary, the Smart Home Assistant using ESP32 and voice commands demonstrates how IoT technology can be effectively used to transform conventional homes into intelligent, automated environments, enhancing both quality of life and energy efficiency.

1.2 Problem Statement

Road In the present day, most household electrical appliances are controlled manually using conventional wall switches. This method is not only time-consuming but also inefficient, especially when users need to operate multiple devices frequently. In many situations, people forget to turn off appliances such as lights, fans, or other electronic devices, which leads to unnecessary power consumption and increased electricity bills. This highlights the need for a more intelligent and automated system to manage home appliances efficiently.

Another significant issue is accessibility. Elderly individuals, physically disabled persons, or patients with limited mobility often face difficulty in reaching and operating switches. This limits their independence and comfort within their own homes. A system that can be controlled through simple voice commands can greatly improve their quality of life by reducing physical effort and enabling hands-free operation.

Although there are several smart home automation systems available in the market, they come with certain drawbacks. Most commercial solutions are expensive and not affordable for middle-class families. They may also require complex installation procedures, additional hardware components, and technical expertise, making them less practical for everyday users. Furthermore, some systems rely heavily on proprietary ecosystems, limiting flexibility and customization.

There is also a lack of simple, low-cost solutions that effectively combine wireless communication and voice control in a single system. Many existing systems either focus only on mobile app control or require multiple integrations for voice functionality, increasing complexity.

To overcome these challenges, this project proposes the development of a smart home assistant using the ESP32. The ESP32 provides built-in Wi-Fi capability, making it ideal for IOT-based applications without the need for additional communication modules. The system will allow users to control home appliances through voice commands using platforms like Google Assistant or mobile applications such as Blynk.

The proposed solution aims to be:

- **Cost-effective**, making it accessible to a wide range of users
- **User-friendly**, requiring minimal technical knowledge
- **Efficient**, reducing energy wastage
- **Accessible**, especially for elderly and disabled individuals
- **Flexible and scalable**, allowing future enhancements

1.3 Research Motivation

The rapid advancement of the Internet of Things (IoT) has created new opportunities to transform traditional homes into intelligent and automated environments. One of the major motivations behind this project is the growing demand for smart home technologies that improve convenience, efficiency, and quality of life. Despite the availability of advanced systems, there is still a gap between high-cost commercial solutions and affordable, user-friendly alternatives suitable for everyday users.

A key motivation for this research is the need to simplify human interaction with technology. Traditional methods of controlling appliances require physical effort and constant user involvement. By integrating voice control through platforms like Google Assistant, this project aims to create a natural and intuitive way of interaction, where users can operate devices using simple spoken commands. This aligns with the trend of human-centered design, where technology adapts to users rather than the other way around.

Another important driving factor is accessibility and inclusivity. Many individuals, such as elderly people and those with physical disabilities, face challenges in performing routine tasks like switching appliances on or off. A voice-controlled smart home system eliminates physical barriers and enables independent living. This social impact is a strong motivation for developing such systems.

Energy efficiency is also a major concern in modern households. Wastage of electricity due to negligence or lack of monitoring is a common problem. Smart automation systems can help reduce energy consumption by allowing users to control appliances remotely and ensure they are used only when needed. This not only reduces electricity bills but also contributes to environmental sustainability.

From a technical perspective, the availability of low-cost and powerful microcontrollers like the ESP32 has made it easier to develop IoT-based solutions. The ESP32 offers built-in Wi-Fi, sufficient processing power, and flexibility, making it an ideal choice for implementing real-time smart home systems. This motivates researchers and students to explore innovative applications without requiring expensive hardware.

1.4 Applications

1. Home Automation

- Control lights, fans, and appliances using voice
- Provides comfort and convenience in daily life
- Uses ESP32 for wireless control

2. Assistive Technology

- Helps elderly and physically disabled individuals
- Enables hands-free operation through voice assistants like Google Assistant
- Improves independence and accessibility

3. Energy Management Systems

- Reduces power wastage by controlling devices efficiently
- Allows remote switching OFF appliances when not in use
- Helps in lowering electricity bills

4. Smart Offices

- Automates lighting, fans, and other office equipment
- Enhances productivity and reduces manual effort
- Can be integrated with scheduling systems

5. Hotel and Hospitality Industry

- Used in smart rooms for voice-controlled services
- Improves customer experience and luxury feel
- Automates room appliances

6. Healthcare Applications

- Used in hospitals for patient assistance systems
- Enables patients to control devices without movement
- Useful for bedridden patients

7. Educational Projects and Research

- Used for learning IOT, embedded systems, and automation
- Helps students understand real-time applications

8. Smart Cities Development

- Can be extended to control street lights and public systems
- Contributes to modern urban infrastructure

CHAPTER – 2

CHAPTER -2: LITERATURE SURVEY

2.1 Overview of Home automation system

Existing home automation systems are designed to simplify the control and monitoring of household appliances by integrating various technologies such as wireless communication, sensors, and internet connectivity. These systems allow users to operate devices like lights, fans, air conditioners, security cameras, and other appliances either remotely or automatically, reducing manual effort and improving convenience.

Traditionally, home automation began with wired systems, where all devices were connected through physical cables to a central controller. While these systems are reliable and secure, they are expensive and difficult to install, especially in already constructed buildings. With the advancement of technology, wireless home automation systems have become more popular. These systems use communication technologies such as Wi-Fi, Bluetooth, Zigbee, and Z-Wave to connect devices, making installation easier and more flexible.

Modern home automation systems are mostly based on IOT (Internet of Things), where devices are connected to the internet and can be controlled from anywhere using smartphones or computers. Many systems also support voice control through assistants like Google Assistant and Amazon Alexa, allowing users to interact with devices using simple voice commands. This adds a layer of convenience and makes the system more user-friendly.

These systems often include features such as scheduling, where devices can be programmed to turn on or off at specific times, and sensor-based automation, where actions are triggered by environmental conditions like motion, temperature, or light. For example, lights can automatically turn on when motion is detected, or fans can operate based on room temperature.

Despite their advantages, existing home automation systems have certain limitations. Many commercial solutions are expensive and not affordable for all users. They may require complex setup procedures and technical knowledge, making them difficult for beginners to use. Additionally, most systems depend heavily on continuous internet connectivity, and any network failure can disrupt their operation. There are also concerns related to data privacy and security, as many systems store user data on cloud platforms.

2.2 Voice Controlled Systems

Voice-controlled systems are an advanced form of human-machine interaction that allow users to operate devices using spoken commands instead of physical controls. In this project, voice control is the key feature that enables users to manage home appliances in a simple, fast, and hands-free manner.

The system works by capturing the user's voice through a smartphone or a voice-enabled platform. The spoken command is processed using voice recognition technology provided by assistants such as Google Assistant. These platforms convert human speech into text and interpret the intended action. Once the command is recognized, it is transmitted over the internet to the microcontroller.

At the core of this system is the ESP32, which receives the command via Wi-Fi. The ESP32 processes the received data and sends signals to the connected relay module. Based on the command, the relay switches the respective appliance ON or OFF. For example, when the user says "Turn on the light," the system recognizes the command and activates the relay connected to the light.

2.3 Comparison of Technologies

In the field of home automation, several technologies are commonly used, each with its own advantages and limitations. Understanding these technologies helps in selecting the most suitable platform for a smart home assistant system.

ESP32 Microcontroller: The ESP32 is a low-cost, powerful microcontroller with built-in Wi-Fi and Bluetooth. It is capable of processing voice commands and controlling appliances through relay modules. Its low cost, ease of integration, and support for IoT applications make it an ideal choice for smart home projects. However, it has limited GPIO pins compared to larger boards, which may restrict the number of connected devices in very large setups.

Bluetooth-Based Systems: These systems use Bluetooth to communicate between devices and controllers. Bluetooth is low-power and easy to implement for local automation. However, its range is limited (typically up to 10 meters), and it cannot provide remote control over the internet, which is a significant limitation for IoT-based smart homes.

Voice Assistant Integration: Platforms like Google Assistant and Amazon Alexa provide cloud-based voice recognition services. They allow hands-free operation of devices using natural language. While highly accurate and convenient, these systems require an internet connection and raise potential privacy concerns due to cloud-based processing.

Relay Modules: Relay modules act as electrically operated switches, allowing microcontrollers to control high-voltage appliances safely. They are simple, cost-effective, and reliable but provide only on/off functionality without any intelligent control on their own.

2.4 Limitations of Existing System

Although existing home automation systems provide convenience, efficiency, and advanced features, they also have several limitations that make them less practical for everyday users.

These limitations are the main reasons for developing a simpler, cost-effective, and voice-controlled system using ESP32.

1. High Cost

- Many commercial smart home systems are expensive, requiring specialized hardware, sensors, and proprietary software.
- High costs make them inaccessible to middle-class households and small-scale applications.
-

2. Complex Installation and Setup

- Existing systems often require professional installation and technical expertise.
- Setting up devices, connecting them to networks, and configuring automation rules can be difficult for non-technical users.
-

3. Dependence on Internet Connectivity

- Most modern smart home systems rely heavily on Wi-Fi or cloud-based services for operation.
- Any internet disruption can prevent remote access or proper functioning of the system.
-

4. Limited Accessibility for Elderly and Disabled Users

- Many systems still rely on mobile apps, touchscreens, or manual interaction.
- This limits independence for people with physical disabilities or mobility challenges.
-

5. Compatibility Issues

- Devices from different brands may not work together seamlessly.
- Proprietary ecosystems can restrict customization or integration with other platforms.
-

6. Privacy and Security Concerns

- Cloud-based voice assistants and smart devices collect user data.
- Risk of hacking, data breaches, or unauthorized access is a concern for users.
-

7. Limited Flexibility and Scalability

- Many systems cannot be easily expanded to include new appliances.
- Adding or modifying devices may require additional modules or complex reconfiguration.
-

8. Response Time and Reliability

- Cloud-dependent systems may have delays due to network latency.
- Some automation functions fail if the system experiences technical glitches.

CHAPTER - 3

CHAPTER -3: EXISTING SYSTEM

3.1 Existing System

Existing home automation systems are designed to control and monitor household appliances using various technologies. These methods can be broadly classified into manual, wired, wireless, and IoT-based systems. Each method has its own characteristics, advantages, and limitations.

1. Manual Control Systems

The most basic method, where devices are controlled directly via switches, knobs, or remote controls.

Examples: Wall switches for lights, remote-controlled fans or AC units.

Limitations:

Requires physical presence to operate appliances.

Not convenient for multiple devices or large homes.

No automation or remote access.

2. Wired Home Automation Systems

Devices are interconnected through physical wiring and connected to a central controller.

The controller can automate appliances based on pre-set schedules or sensors.

Examples: Centralized lighting control panels in offices or commercial buildings.

Limitations:

Installation is expensive and labor-intensive.

Difficult to modify or expand after installation.

Limited flexibility for adding new devices.

3. Wireless Home Automation Systems

Uses wireless communication protocols like Wi-Fi, Bluetooth, Zigbee, or Z-Wave.

Appliances can be controlled via mobile apps or remote devices.

Examples: Smart bulbs, Wi-Fi-enabled fans, and smart plugs.

Limitations:

Limited range (especially for Bluetooth-15 -30 m).

Dependent on stable network connectivity.

May face interference from other wireless devices.

4. IOT-Based Smart Home Systems

Integrates devices with the internet for remote control and automation.

Voice assistants like Google Assistant or Amazon Alexa are used for hands-free control.

Sensors (motion, temperature, light) and cloud services automate operations.

Examples: Smart thermostats, automated lighting systems, smart security cameras.

Limitations:

Requires constant internet connectivity.

Setup can be complex for non-technical users.

Privacy and security concerns due to cloud-based data storage.

5. Mobile App-Based Systems

Users control appliances through smartphone apps like Blynk, Smart Things, or Home Assistant.

Limitations:

Users need a smartphone or tablet to operate devices.

Not fully hands-free; physical interaction is still required.

3.2 Limitations of Existing System

1.High Cost of Installation and Devices – Commercial systems are expensive and not affordable for all users.

2.Limited Accessibility – Difficult for elderly or physically disabled users to operate without voice or automation.

3.Compatibility Issues – Devices from different brands may not work seamlessly together.

4.Privacy and Security Concerns – Cloud-based systems can expose user data to unauthorized access.

5.Complex Setup and Configuration – Non-technical users often struggle with installation and programming

Existing home automation methods provide convenience but often fail in terms of cost-effectiveness, ease of use, accessibility, and flexibility. These limitations create the need for a low-cost, ESP32-based, voice-controlled smart home system that is simple to set up, hands-free, scalable, and accessible to all users, including elderly and differently-abled individuals.

COMPARISON TABLE: EXISTING SYSTEM vs PROPOSED SYSTEM

Feature	Existing System	Proposed System
Control Method	Manual switches	Voice control via Android app
Communication	Wired	Wireless Bluetooth
Microcontroller	Conventional MCU	ESP32
Bluetooth	External module required	Inbuilt Bluetooth
User Convenience	Low	Very High
Accessibility	Difficult for disabled users	Highly accessible
Appliance Feedback	Not available	LCD display
Load Types	Mostly AC only	AC and DC loads
Scalability	Limited	Easily expandable
Cost	Higher (extra modules)	Lower
Power Efficiency	Moderate	Improved
Smart Features	Not supported	Supported

CHAPTER - 4

CHAPTER -4: PROPOSED SYSTEM

4.1 Proposed System

The proposed system aims to design and implement a Smart Home Automation Assistant using the ESP32 microcontroller integrated with voice recognition technology. The system enables users to control home appliances such as lights, fans, and other electrical devices through voice commands, providing convenience, energy efficiency, and enhanced accessibility.

System Overview

The system consists of an ESP32 module connected to various sensors and actuators, along with a voice input interface. The user gives voice commands through a mobile application or a voice recognition module, which are then processed and transmitted to the ESP32 via Wi-Fi or Bluetooth. Based on the received command, the ESP32 controls the corresponding appliances through relay modules.

Key Components

- **ESP32 Microcontroller** – Acts as the central processing unit with built-in Wi-Fi and Bluetooth.
- **Voice Recognition System** – Uses mobile apps (like Google Assistant) or dedicated modules to capture and interpret voice commands.
- **Relay Module** – Controls electrical appliances (ON/OFF switching).
- **Sensors (optional)** – Temperature, motion, or light sensors for automation.
- **Mobile Application / IOT Platform** – Interface for user interaction and remote access.

Working Principle

1. The user gives a voice command (e.g., “Turn on the light”).
2. The command is processed using a voice recognition platform.
3. The processed command is sent to the ESP32 over Wi-Fi.
4. ESP32 interprets the command and triggers the appropriate relay.
5. The appliance is turned ON/OFF accordingly.
6. Optional sensors can automate actions (e.g., lights turn on when motion is detected).

Features of the Proposed System

- Voice-controlled home automation
- Remote access via internet
- Real-time monitoring and control

- Low-cost and energy-efficient design
- Expandable system for additional devices
- User-friendly interface

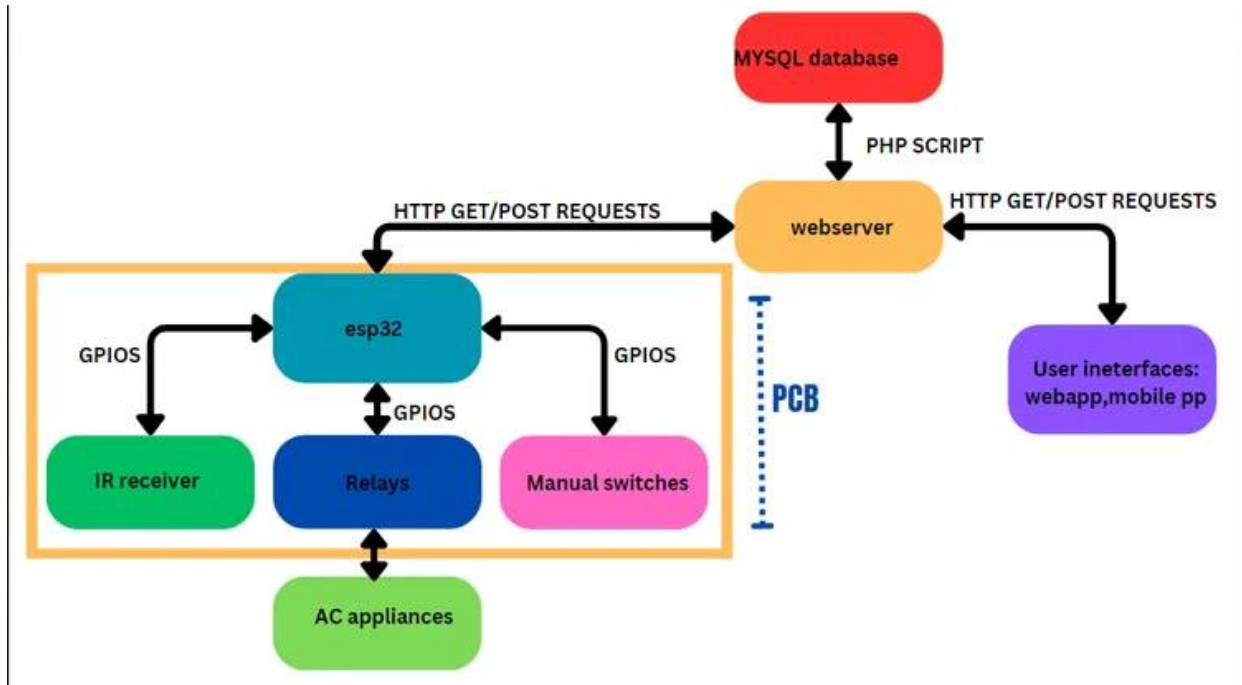


Figure 4.1: System Architecture of smart home Assistant using ESP-32 & voice commands

4.2 Methodology

The development of the Smart Home Assistant using ESP32 and Voice Commands follows a systematic approach involving hardware integration, software development, and communication setup. The methodology is divided into the following stages:

1. System Design

The overall architecture of the system is designed by identifying key components such as the ESP32 microcontroller, relay modules, voice input interface, and optional sensors. A block diagram is created to illustrate the interaction between these components.

2. Hardware Setup

- The ESP32 is interfaced with relay modules to control electrical appliances.
- Appliances like lights and fans are connected to the relay outputs.
- Optional sensors (e.g., motion or temperature sensors) are connected to GPIO pins.
- Proper power supply and safety measures are ensured during circuit setup.

3. Voice Command Integration

- Voice commands are captured using a mobile application or voice assistant (e.g., Google Assistant).
- Commands are processed using a cloud-based or app-based voice recognition system.
- Each voice command is mapped to a specific action (e.g., turning ON/OFF devices).

4. Communication Setup

- The ESP32 is connected to Wi-Fi for internet-based communication.
- A communication protocol such as HTTP, MQTT, or Firebase is used to send and receive data.
- The processed voice command is transmitted from the app/cloud to the ESP32.

5. Software Development

- Embedded C/C++ code is written using the Arduino IDE for ESP32.
- The program includes:
 - Wi-Fi connectivity setup
 - Command reading and decoding
 - GPIO control for relays
- Logic is implemented to match received commands with appliance actions.

6. Automation Logic (Optional)

- Sensor-based automation is implemented (e.g., lights turn on when motion is detected).
- Conditions and thresholds are defined in the code for automatic control.

7. Testing and Debugging

- Individual modules (Wi-Fi, relays, voice commands) are tested separately.
- Integrated system testing is performed to ensure proper communication.
- Errors and delays are identified and resolved.

8. Implementation and Deployment

- The system is installed in a real-time environment (home setup).
- Final adjustments are made for reliability and user convenience.
- Performance is evaluated based on response time and accuracy.

9. Evaluation

- The system is evaluated for:
 - Response time to voice commands
 - Accuracy of command execution
 - Power consumption
 - User satisfaction

4.3 Workflow

The proposed smart home assistant system operates by integrating voice recognition technology with the ESP32 microcontroller to enable seamless control of household appliances. The workflow begins when the system is powered on, and the ESP32 initializes all required components such as GPIO pins, relay modules, and communication interfaces.

Once initialized, the ESP32 establishes a connection to a Wi-Fi network, allowing it to communicate with cloud services or mobile applications. This connection is essential for receiving voice commands and sending feedback to the user.

The user interacts with the system by giving voice commands through a voice-enabled application or assistant. These voice inputs are processed using a voice recognition system, which converts speech into text and maps it to predefined commands such as turning appliances ON or OFF.

After processing, the command is transmitted over the internet using a suitable communication protocol like HTTP or MQTT. The ESP32 continuously monitors incoming data and receives the command in real time.

Upon receiving the command, the ESP32 validates and interprets it to determine the required action. If the command is valid, the controller activates the corresponding GPIO pin connected to the relay module. The relay then switches the electrical appliance ON or OFF as per the instruction.

In addition, the system can optionally send feedback to the user interface, confirming whether the action has been successfully executed. This improves reliability and user experience.

The entire process runs in a continuous loop, allowing the system to respond to multiple commands efficiently.

Step-by-Step Workflow

1. System Initialization

- The ESP32 microcontroller is powered on.
- GPIO pins, relay modules, and peripherals are initialized.

2. Wi-Fi Connection Establishment

- ESP32 connects to the configured Wi-Fi network.
- Ensures stable internet/Bluetooth connectivity for voice communication.
-

3. Voice Command Input

- The user gives a voice command through a mobile app (e.g., Arduino Bluetooth control app).
-

4. Voice Processing

- The voice command is converted into text using a cloud-based voice recognition system.
- The processed command is mapped to a predefined action.
-

5. Command Transmission

- The command is sent to the ESP32 via an IoT platform (e.g., MQTT/HTTP/Firebase).
-

6. Command Reception by ESP32

- ESP32 continuously listens for incoming data.
- Receives the command from the server or application.
-

7. Command Validation and Processing

- The received command is checked for validity.
- If valid, it is decoded to identify the target device and action.
-

8. Device Control Execution

- ESP32 sends signals to the relay module.
- The respective appliance (light, fan, etc.) is turned ON/OFF.
-

9. Feedback Mechanism (Optional)

- The system sends status feedback (e.g., device ON/OFF) back to the app.

10. Continuous Monitoring Loop

- The system keeps running in a loop.
- Waits for the next voice command or sensor input.

4.4 Advantages

1. Hands-Free Operation

The system allows users to control appliances using voice commands, eliminating the need for manual switches and making it very convenient, especially when hands are occupied.

2. Low Cost and Affordable

The ESP32 microcontroller is inexpensive compared to other automation systems, making the overall setup cost-effective and suitable for students and small-scale applications.

3. Wireless Connectivity

With built-in Wi-Fi and Bluetooth, ESP32 enables seamless wireless communication without requiring additional modules, reducing complexity and cost.

4. Remote Access

Users can control home appliances from anywhere using a smartphone and internet connection, providing flexibility and convenience.

5. Energy Efficiency

Appliances can be turned ON/OFF as needed, preventing unnecessary power consumption and helping reduce electricity bills.

6. Easy Installation and Maintenance

The system is simple to install and does not require major changes to existing electrical wiring. Maintenance is also minimal.

7. Scalability and Expandability

Additional devices and sensors can be easily added to the system, allowing future expansion of the smart home setup.

8. Improved Accessibility

Voice control is highly beneficial for elderly and physically disabled individuals, enabling them to operate appliances easily.

9. Real-Time Control and Monitoring

The system provides instant response to commands and can offer feedback on device status, improving user experience.

10. Integration with IOT Platforms

The system can be integrated with various IOT platforms (like MQTT, Firebase), enabling advanced automation and smart features.

11. Customization

Users can customize commands and automation rules according to their preferences and needs.

CHAPTER - 5

CHAPTER -5: BLOCK DIAGRAM

5.1 BLOCK DIAGRAM

The block diagram of the smart home automation system illustrates the interaction between various components used to control household appliances through voice commands.

The system begins with the user, who provides input in the form of voice commands such as turning appliances ON or OFF. These commands are captured using a **voice assistant or mobile application**, which converts the spoken words into text using voice recognition technology.

Once the command is processed, it is transmitted over the **internet via Wi-Fi** to the ESP32 microcontroller. The ESP32 acts as the central control unit of the system, receiving and interpreting the command.

After receiving the command, the ESP32 processes it and determines the appropriate action. It then sends control signals to the **relay module**, which acts as an electronic switch. The relay module is responsible for controlling the power supply to the connected electrical appliances.

The **home appliances**, such as lights, fans, or other devices, are connected to the relay outputs. Based on the command, the relay turns the appliances ON or OFF accordingly.

Additionally, **sensors** such as motion sensors or temperature sensors can be connected to the ESP32 to enable automatic control. These sensors provide real-time environmental data, allowing the system to make intelligent decisions, such as turning on lights when motion is detected or activating a fan when the temperature rises.

The entire system operates continuously, allowing the user to control appliances remotely and efficiently through voice commands. This integration of voice technology, wireless communication, and embedded systems makes the solution smart, user-friendly, and energy-efficient.

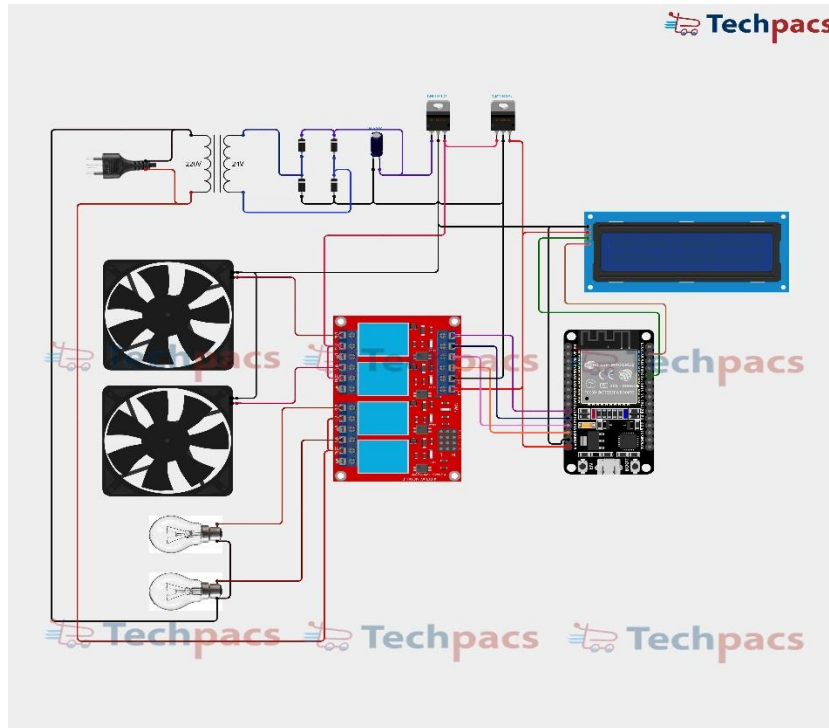


Figure-5.1: Block Diagram of Smart Home Assistant using ESP-32 & Voice Commands

5.2 Overall System Design

The overall system design of the smart home automation system integrates voice recognition, wireless communication, and embedded control to provide an efficient and user-friendly solution for controlling household appliances.

1. System Architecture

The system follows a client-server architecture:

- The **client side** consists of the user interface (mobile app or voice assistant), where the user gives voice commands.
- The **server/cloud** processes the voice input and converts it into a digital command.
- The **ESP32 microcontroller** acts as the main controller on the hardware side, executing the commands.

2. Input Layer (Voice Interface)

- The user interacts with the system using voice commands.
- A voice assistant or mobile application captures the speech input.
- The speech is converted into text using a voice recognition system.
- Commands are predefined (e.g., “Light ON”, “Fan OFF”).

3. Communication Layer

- The processed command is transmitted via **Wi-Fi / Bluetooth**.
- Communication protocols such as **HTTP, MQTT, or Firebase** are used.
- This layer ensures reliable data transfer between the user interface and ESP32.

4. Control Layer (ESP32)

- ESP32 acts as the brain of the system.
- It receives commands from the cloud/server.
- The controller decodes the command and identifies the target device.
- Based on logic, it activates the corresponding GPIO pins.

5. Output Layer (Actuators)

- The relay module is connected to ESP32 output pins.
- Relays function as electronic switches to control appliances.
- Devices like lights, fans, and sockets are connected to relays.
- Appliances are turned ON/OFF based on commands.

6. Sensor Layer

- Sensors such as motion, temperature, or light sensors can be integrated.
- These sensors provide real-time data to ESP32.
- Automation rules can be applied (e.g., auto light ON when motion detected).

7. Feedback Mechanism

- The system can send feedback to the user via the app.
- Displays appliance status (ON/OFF).

8. Power Supply Design

- ESP32 operates on low voltage (3.3V).
- Relays and appliances use external power supply.
- Proper voltage regulation and isolation are maintained for safety.

9. System Operation Flow

Voice Command → Processing → Wi-Fi Transmission → ESP32 → Relay → Appliance Control → Feedback

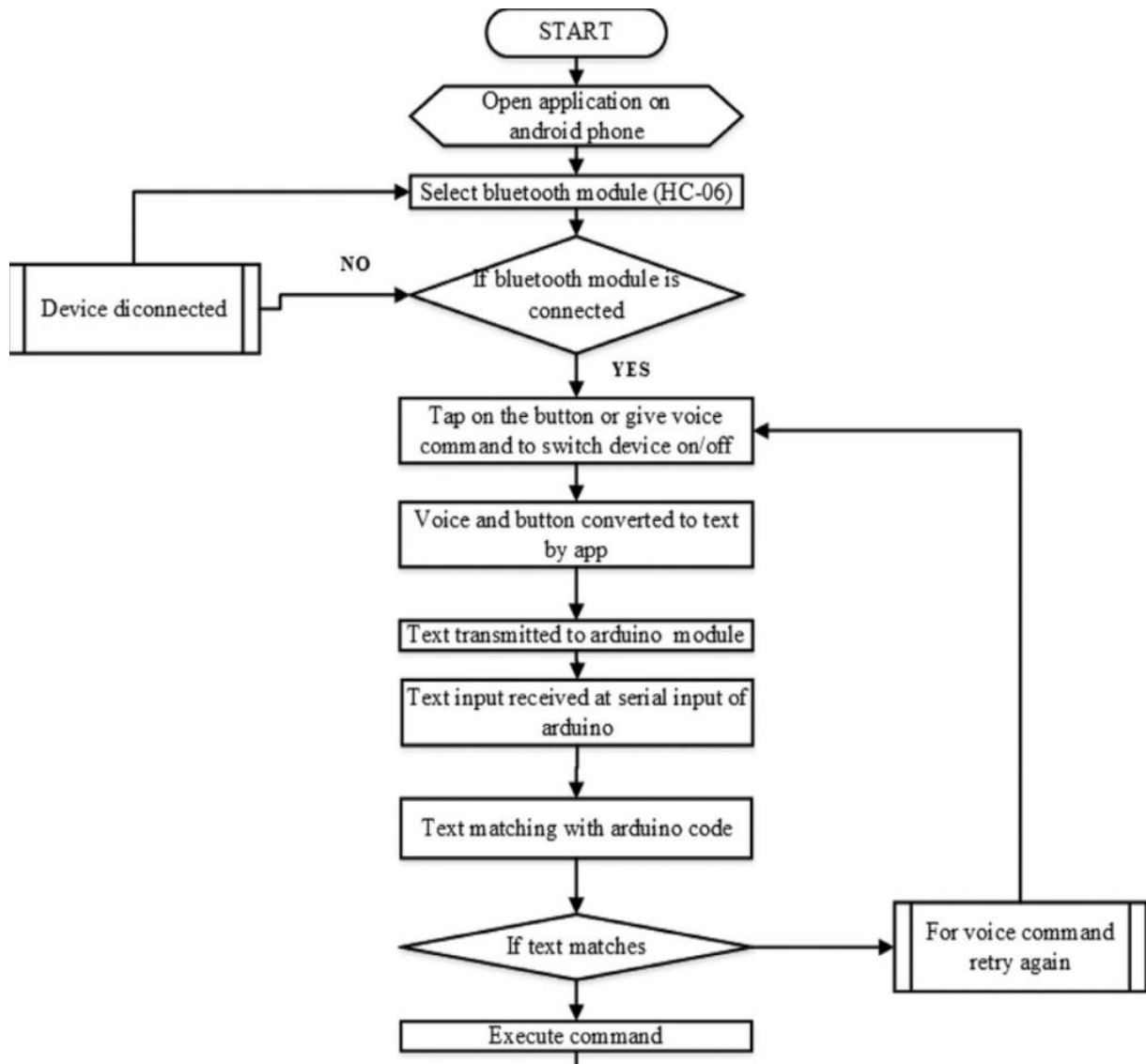


Figure-5.2: Overall Design Of Smart Home Automation System

5.3 Working Principle

The smart home automation system works on the principle of **voice-controlled wireless communication using the ESP32 microcontroller**. It enables users to control electrical appliances through voice commands, which are processed and transmitted over the internet to execute the desired actions.

Initially, when the system is powered ON, the ESP32 microcontroller initializes all connected components such as relay modules and sensors. It then connects to a Wi-Fi network, allowing it to communicate with external devices and cloud platforms.

The user gives a voice command through a mobile application or voice assistant, such as **Google Assistant**. The voice input is captured and converted into text using a speech recognition system. This processed command is then mapped to a predefined instruction (for example, turning a light ON or OFF).

The command is transmitted via the internet using communication protocols like HTTP or MQTT. The ESP32 continuously monitors incoming data and receives the command in real time.

Once the command is received, the ESP32 processes and interprets it. Based on the decoded instruction, it activates the corresponding GPIO pin connected to a relay module. The relay acts as a switch, controlling the electrical circuit of the appliance.

As a result, the appliance (such as a light or fan) is turned ON or OFF according to the user's command. Optionally, the system can send feedback to the user interface to confirm the action.

Additionally, if sensors are integrated, the system can operate automatically by detecting environmental conditions (such as motion or temperature) and controlling appliances without manual input.

Thus, the system provides a seamless, efficient, and user-friendly method of controlling home appliances using voice commands

5.4 Data flow diagram

A data flow diagram (DFD) is a graphical representation of how data moves within an information system. It is a modelling technique used in system analysis and design to illustrate the flow of data between various processes, data stores, data sources, and data destinations within a system or between systems.

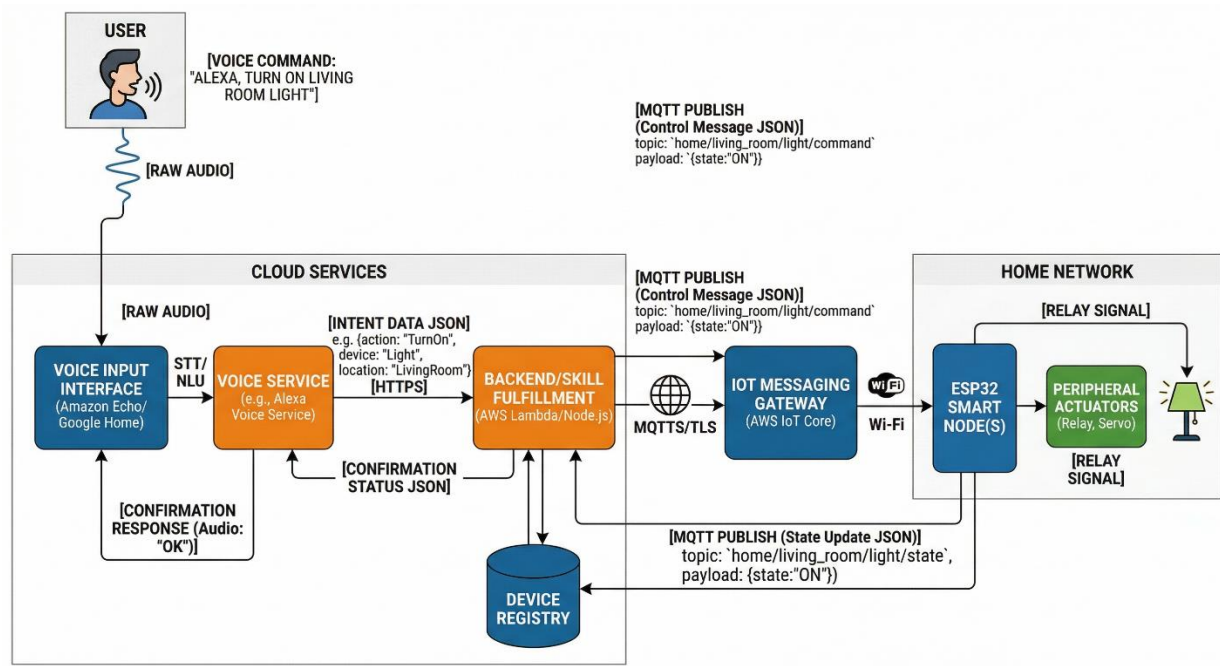


Figure-5.4: Data flow Diagram

5.5 Component diagram

Component diagram describes the organization of the physical components in a system.

1. ESP32 Microcontroller

- **Description:** The ESP32 is a powerful, low-cost microcontroller with built-in Wi-Fi and Bluetooth connectivity, making it ideal for IOT projects.
- **Function in Project:** Acts as the central control unit, receiving commands from the user and controlling appliances through relay modules. It also reads sensor inputs for automation.
- **Key Features:** Dual-core processor, multiple GPIO pins, low power consumption, supports PWM, ADC, and communication protocols like MQTT/HTTP.

2. Relay Module

- **Description:** An electromagnetic switch that allows a low-voltage signal from the ESP32 to control high-voltage appliances safely.
- **Function in Project:** Acts as a bridge between the microcontroller and household appliances. Turns appliances ON/OFF based on ESP32 signals.
- **Key Features:** Provides electrical isolation, fast switching, and can handle high-current appliances.

3. Home Appliances

- **Description:** Devices such as lights, fans, or other electrical equipment connected to the smart home system.
- **Function in Project:** Receives control signals via the relay module and executes actions (turning ON/OFF).
- **Key Features:** Can be any standard electrical appliance; response depends on relay switching speed.

4. Voice Recognition / Mobile Application

- **Description:** Software platform (like Google Assistant or a custom app) that converts spoken commands into digital instructions.
- **Function in Project:** Captures user voice commands and sends them to the ESP32 over Wi-Fi.
- **Key Features:** Real-time speech-to-text conversion, command mapping, remote accessibility.

5. Sensors

- **Description:** Electronic devices that detect environmental parameters.
- **Function in Project:** Provide data to ESP32 for automation (e.g., lights automatically turn ON when motion is detected).
- **Key Features:** Motion sensors, temperature sensors, or light sensors; fast response and low power consumption.

6. Power Supply

- **Description:** Provides electrical power to the ESP32 and relay modules.
- **Function in Project:** ESP32 operates on 3.3V, while relays and appliances may require higher voltages. Proper voltage regulation ensures safety and system stability.
- **Key Features:** Stable voltage output, safety isolation, and sufficient current rating for all connected devices.

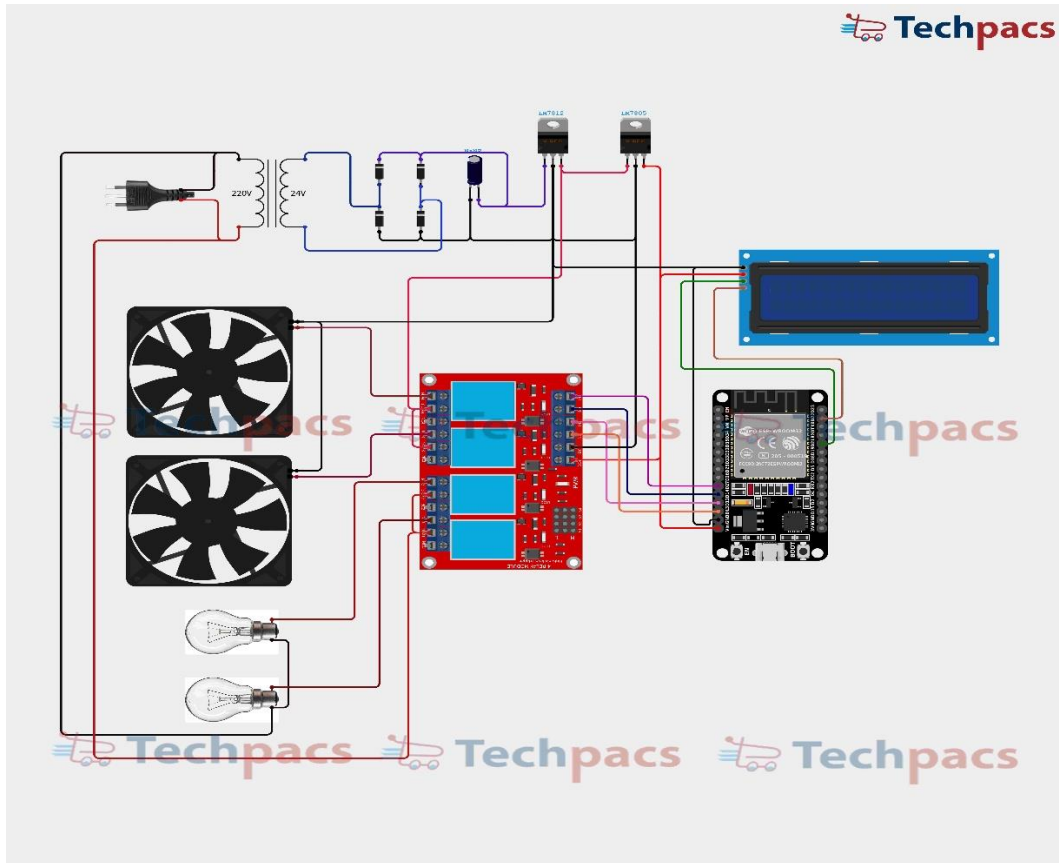


Figure-5.5: Component Diagram of Smart Home Automation Using Voice Commands

CHAPTER - 6

CHAPTER -6: HARDWARE COMPONENTS

6.1 ESP-32 Micro Controller

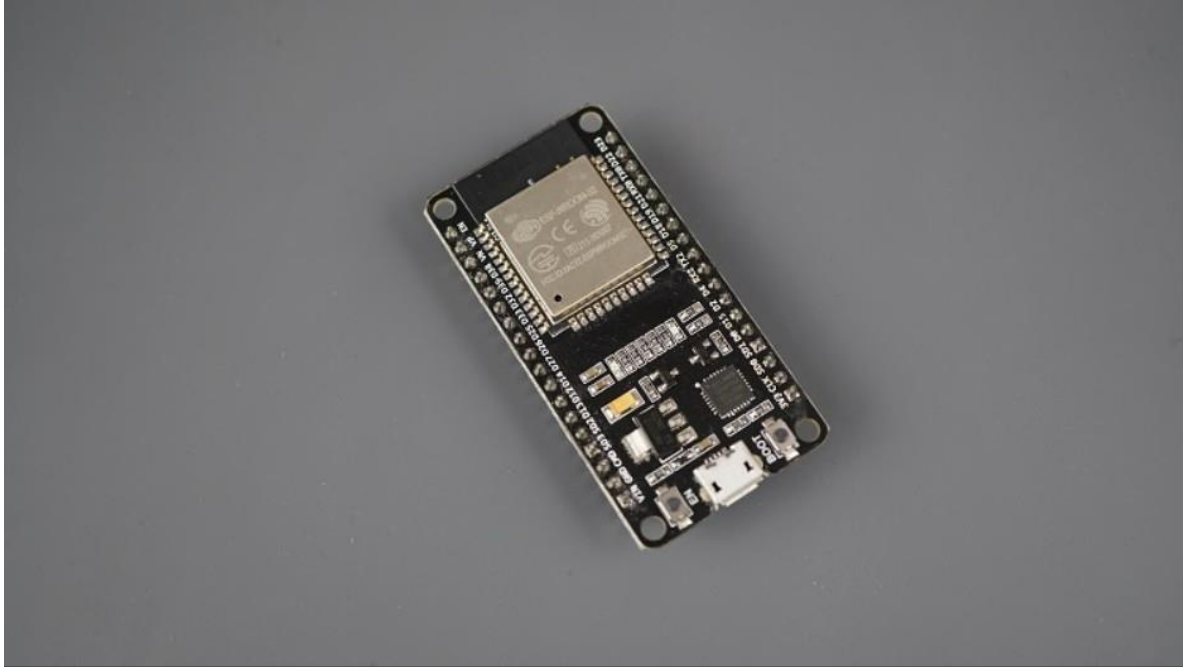


Fig : ESP-32 WROOM Micro Controller

The ESP32-WROOM is a compact, low-cost, and highly integrated Wi-Fi and Bluetooth-enabled microcontroller module. It contains the ESP32 microcontroller chip, built-in flash memory, and an antenna, making it a ready-to-use solution for IoT and smart home applications.

In this project, the ESP32-WROOM acts as the central processing unit (brain) of the smart home system. It receives commands from the user through a mobile application or voice assistant, processes these commands, and controls home appliances via relay modules. It can also interface with sensors to enable automation based on environmental conditions such as motion or temperature.

The module supports multiple communication protocols (like HTTP, MQTT, SPI, I2C) and offers several GPIO pins for connecting actuators and sensors. Its built-in wireless connectivity, high processing speed, and low power consumption make it ideal for real-time, reliable, and energy-efficient smart home automation systems.

6.2 RELAY MODULE

A relay module is an electromechanical switch that allows a low-voltage microcontroller like the ESP32 to control high-voltage electrical appliances safely. It acts as a bridge between the microcontroller and devices such as lights, fans, or other home appliances.

In this project, the relay module performs the following functions:

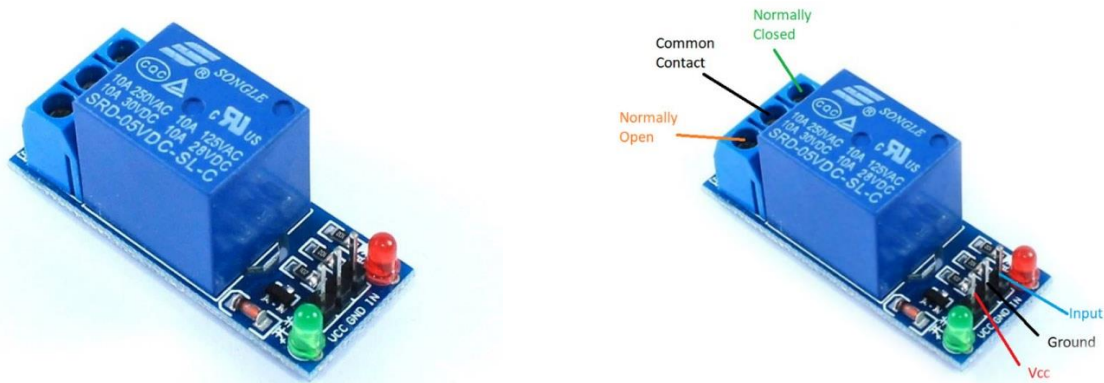


Fig : Relay Modules

1. Signal Control:

- Receives digital signals from the ESP32's GPIO pins.
- Each signal corresponds to turning a connected appliance ON or OFF.
-

2. Electrical Isolation:

- Separates the low-voltage ESP32 circuit from high-voltage AC appliances, ensuring **safety** for both the device and the user.
-

3. Actuation of Appliances:

- When the ESP32 sends a HIGH/LOW signal, the relay coil is energized or de-energized.
- This mechanically switches the connected appliance ON or OFF.
-

4. Performance in the Project:

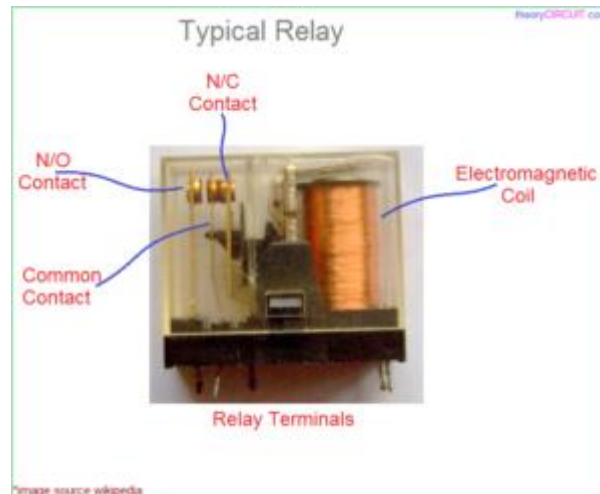
- Provides **fast and reliable switching** of appliances.
- Supports multiple channels (single-channel or multi-channel relays) for controlling more than one device.

SPDT Relay & DPDT Relay

Relay is a electro-mechanical switch used to control high power application through low power signal electronic circuits, for an example a simple timer circuit working under 5V DC

bias can not control high voltage light bulb, by introducing Relay component we can easily control light bulb.

Typical Relay



Typical Relay contains Electromagnetic coil, N/O contact, N/C contact and Common contact, electromagnetic coil accepts low voltage DC bias and becomes electromagnet to attract the common terminal lever and interchanges the connection between N/C, N/O contacts.

Types of Relay

Different types of Electrical Relay Switches are available in market

1. SPST – Single Pole Single Throw
2. DPST – Double Pole Single Throw
3. SPDT – Single Pole Double Throw
4. DPDT – Double Pole Double Throw

but most frequently used Relays are SPDT Relay and DPDT Relay both are acts as Electromechanical switch.

SPDT Relay

Single Pole Double Throw (SPDT) Relay contains two coil terminals and common terminal, then two switching terminals N/O (Normally Open), N/C (Normally Close)



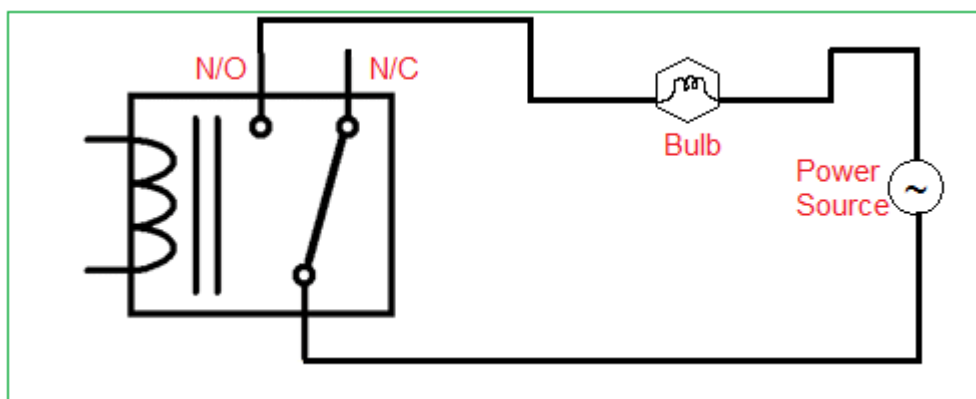
If there is not enough DC supply in coil terminals then Relay represents idle condition that is common terminal connected in N/C terminal. When the coil gets required DC supply then coil gets Magnetically Energized and this magnetic flux force attracts common terminal lever which is made of iron and makes the connection to N/O terminal, now the N/C becomes open.

DPDT Relay

Double Pole Double Throw (DPDT) Relay contains two coil terminals, two separate common terminals C1, C2 and two Normally Open (N/O1, N/O2), two Normally Close (N/C1, N/C2) terminals, this Relay makes two different connection and control with one control signal to the coil.



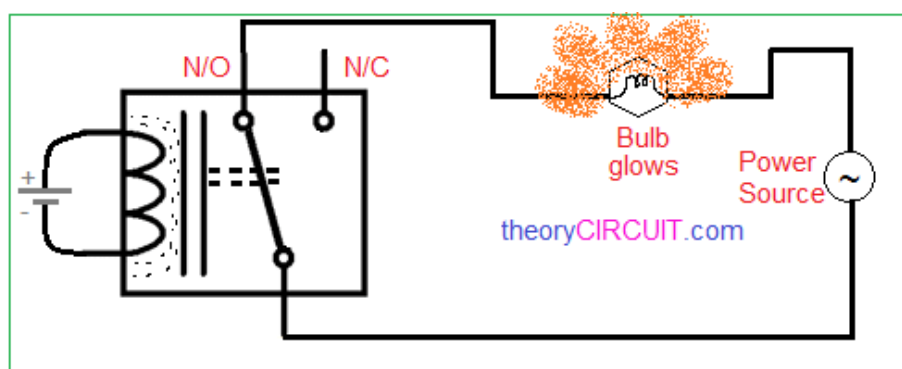
If there is no bias at coil then Relay stays in idle condition that is common terminals are connected with N/C terminals, when DC bias arrives to coil then it gets magnetically energized and attract common terminal levers and makes connection between common



terminals and N/O terminals, now the N/C terminals becomes open and those terminals are works with their pairs.

How the Relay Works?

Here simple relay circuit given for an example, when there is no DC supply the bulb



connected at N/O terminal remains off and When DC supply given to the Relay coil it get magnetically energized and attracts the common terminal lever to N/O terminal and the bulb circuit completes this step makes bulb glow.

6.3 Power Supply Unit

The Power Supply Unit (PSU) provides the necessary electrical power for the smart home automation system to operate efficiently and safely. It converts the main AC voltage from the household supply (usually 110V/220V) into a regulated low DC voltage suitable for the ESP32 microcontroller, relay modules, and other connected components.

Functions in the Project:

1. **Voltage Conversion:**

- Steps down high AC voltage to low DC voltage (typically 5V or 12V) required for relays and the ESP32.

○

2. **Voltage Regulation:**

- Provides a stable and constant voltage to prevent fluctuations that can damage sensitive components like the ESP32.

○

3. **Safety and Isolation:**

- Ensures the high-voltage AC supply is safely isolated from low-voltage electronics.
- Prevents electrical hazards and protects both the microcontroller and connected appliances.

○

4. **Powering Multiple Components:**

- Supplies sufficient current to operate the ESP32 microcontroller, relay modules, and optional sensors simultaneously.
- Supports multi-channel relay modules without causing voltage drops.

REGULATED POWER SUPPLY

In mains-supplied electronic systems the AC input voltage must be converted into a DC voltage with the right value and degree of stabilization. The common DC voltages that are required to power up the devices are generally in the range of 3 VDC to 30 VDC. Typically the fixed types of DC voltages are 5V, 9V, 12V, 15V and 18V DC.

Power supply Modules:

- Step down transformer
- Bridge rectifier with filter
- Voltage regulators

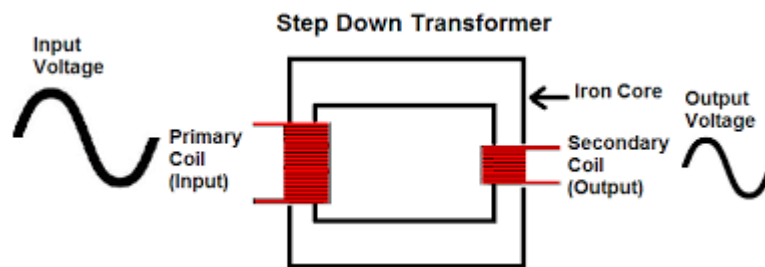
Regulated Power Supply Block diagram:



Transformer

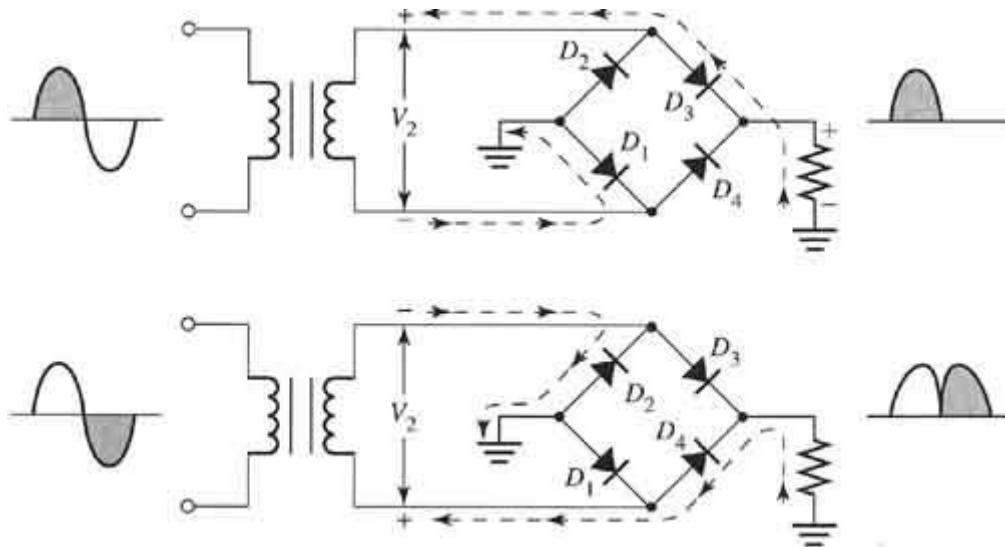
Transformers convert AC electricity from one voltage to another with little loss of power. Transformers work only with AC and this is one of the reasons why mains electricity is AC. Step-up transformers increase voltage, step-down transformers reduce voltage.

A step down power transformer is used to step down the AC voltage from the of 110 VAC or 220 VAC i.e, it converts higher voltage at the input side(V_{in}) to a lower voltage at the output (V_{out}).



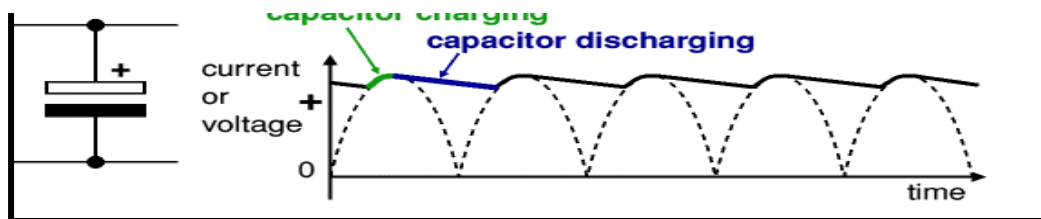
Rectifier

There are several ways of connecting diodes to make a rectifier to convert AC to DC. The bridge rectifier is the most important and it produces **full-wave** varying DC. A bridge rectifier makes use of four diodes in a bridge arrangement to achieve full-wave rectification.



Filter

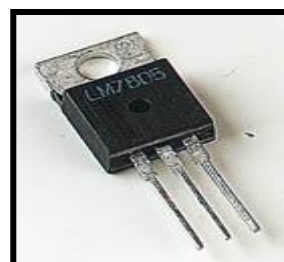
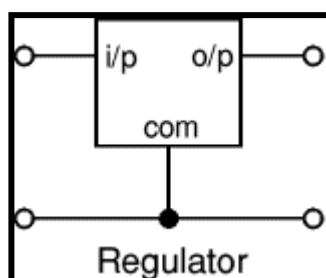
Filtering is performed by a large value electrolytic capacitor connected across the DC supply to act as a reservoir, supplying current to the output when the varying DC



(voltage from the rectifier is falling. The diagram shows the unfiltered varying DC dotted line) and the filtered DC (solid line). The capacitor charges quickly near the peak of the varying DC, and then discharges as it supplies current to the output. Typically 1000 μF capacitor is used

REGULATOR

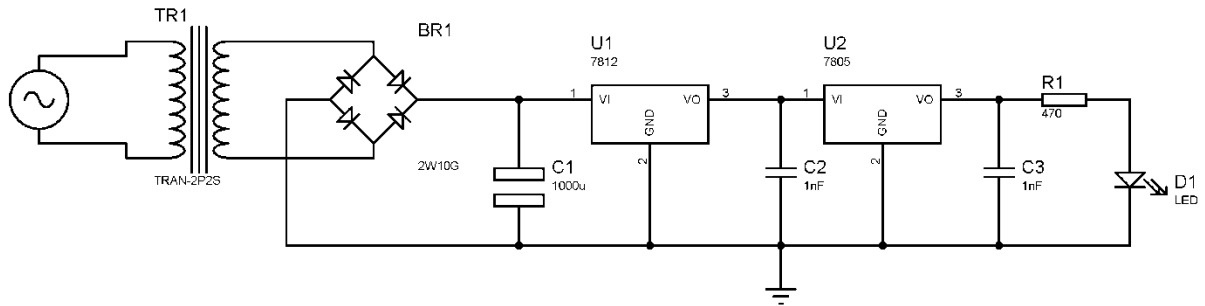
This is a simple DC regulated supply project using 7805 voltage regulator to obtain a variable DC voltage range from 5V to 15V.



Pin out of the 7805 regulator IC.

1. Unregulated voltage in
2. Ground
3. Regulated voltage out

Circuit Diagram of R. P. S



6.4 Electrical Appliances

Electrical appliances are the output devices in a smart home system that are controlled via the ESP32 microcontroller using voice commands. These appliances are connected to the system through relay modules, allowing the low-voltage ESP32 to safely control high-voltage devices.

Common Appliances Used in Smart Home Automation:

. Lights (LED/Bulbs):

Function: Provide illumination in rooms.

Automation: Can be turned ON/OFF via voice commands or sensors

Benefit: Reduces manual switching and saves energy.



. Fans:



Function: Provide air circulation and cooling.

Automation: Can be controlled using voice commands or temperature sensors.

Benefit: Helps maintain comfort and reduces energy wastage.

6.5 connecting wires and circuit setup

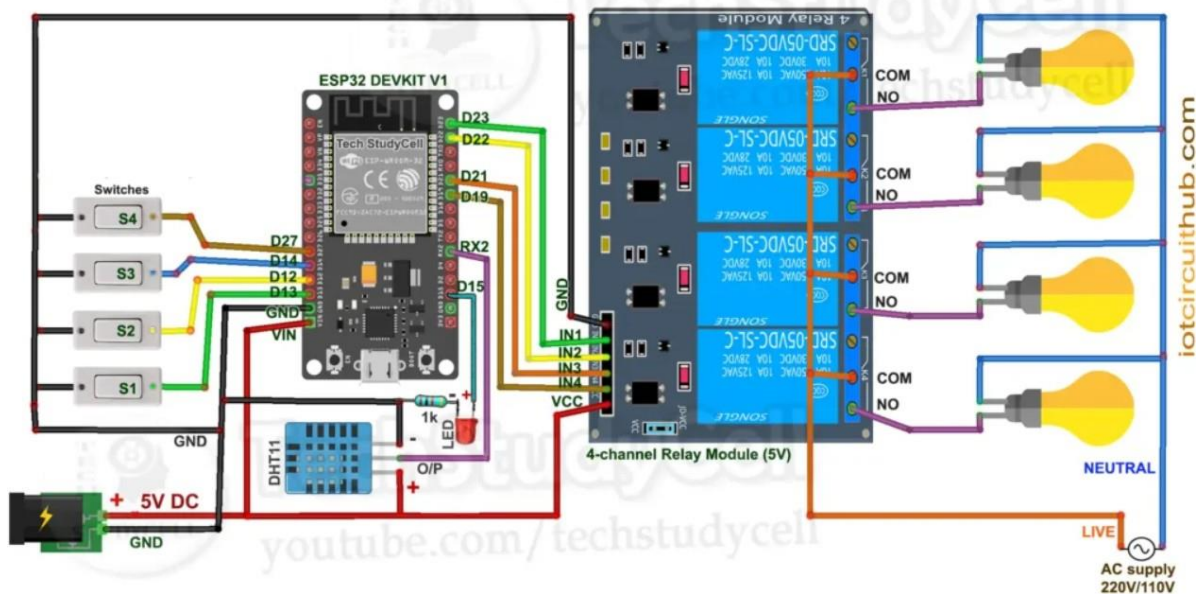


Fig: Connecting Wires in Smart Home Automation using ESP-32

1. Components Needed

- **ESP32-WROOM module** (ESP32 Devkit V1 works fine)
- **Relay module** (to control lights, fans, etc.)
- **Microphone module** (e.g., **INMP441** I2S Mic) or an external voice interface (like Google Assistant / Alexa)
- **Power supply** (5V/3.3V depending on devices)
- Jumper wires and breadboard

2. Basic Wiring Diagram

ESP32 to Relay

- **VCC** → 5V
- **GND** → GND
- **IN1** → GPIO pin on ESP32 (e.g., GPIO 23 for Light 1)
- **IN2** → GPIO pin on ESP32 (e.g., GPIO 22 for Fan 1)

ESP32 to Microphone (I2S)

- **VCC** → 3.3V
- **GND** → GND
- **SD** → GPIO 32 (data)
- **SCK** → GPIO 33 (clock)
- **WS** → GPIO 25 (word select)

7. HARDWARE REQUIREMENTS

Component	Description
ESP32	Main controller with Bluetooth
Relay Module	AC load switching
DC Fan	Low voltage fan
DC Motor	Motor load
Motor Driver (L298N / Relay)	Motor control
16×2 I ² C LCD	Status display
Power Supply	5V / 12V regulated
Android Smartphone	Voice control

CHAPTER - 7

CHAPTER -7: SOFTWARE COMPONENTS

7.1 What is Arduino IDE?



Arduino IDE (Integrated Development Environment) is a software platform used to **write, compile, and upload code** to microcontrollers like **ESP32**.

- It uses C/C++ programming but simplifies many hardware interactions.
- It comes with a built-in library system, making it easy to control sensors, relays, microphones, and other devices.
- Compatible with ESP32 boards, allowing you to program Wi-Fi, Bluetooth, and GPIO functionalities.

Think of Arduino IDE as the “bridge” between your ESP32 hardware and the software logic that will control your smart home devices.

2. Importance of Arduino IDE in Your Project

In your **Smart Home Assistant using ESP32 and voice commands**, Arduino IDE is **essential** for several reasons:

a) Programming the ESP32

- Your ESP32 needs instructions to respond to voice commands, control relays, and connect to Wi-Fi.
- Arduino IDE allows you to **write these instructions**, compile them, and upload them to the ESP32.

b) Easy Integration with Libraries

- Arduino IDE has libraries for:
 - **Wi-Fi and MQTT** (for cloud communication)
 - **I2S Microphone** (for voice input)
 - **Relays and sensors**
- These libraries save time and reduce complex low-level coding.

c) Debugging & Monitoring

- The IDE has a **Serial Monitor**, which shows real-time data from your ESP32.
- Example: You can see if your voice command was recognized or if a relay was triggered correctly.

d) Cross-Platform Support

- Works on Windows, Mac, and Linux.
- Compatible with different ESP32 boards, making your project portable.

e) Community Support

- Arduino IDE has a huge **community and tutorials**, which is invaluable for troubleshooting your smart home project.
- You can find example codes for **ESP32, relays, voice control, and IOT integration**.

7.2 Embedded C Language



1. Definition of Embedded C

Embedded C is a special version of the C programming language designed specifically for **embedded systems** — microcontrollers and small computing devices that perform dedicated tasks.

- Unlike standard C, which is often used for general-purpose applications on PCs, Embedded C is used to **directly control hardware components** like sensors, LEDs, motors, and communication modules.
- It allows **efficient, low-level programming** with minimal memory usage, which is crucial for devices with limited resources.

2. Key Features of Embedded C

1. **Direct hardware access** – you can manipulate registers, input/output pins, and peripherals.
2. **Real-time operation** – suitable for tasks that need immediate response, like reading a microphone or switching a relay.
3. **Efficient memory usage** – critical for microcontrollers with limited RAM and flash.
4. **Portability across microcontrollers** – works with ESP32, Arduino boards, PIC, ARM, and others.

3. Importance in Your Smart Home Assistant Project

For a **Smart Home Assistant** using **ESP32** and **Voice Commands**:

- **Control Devices:** Embedded C lets you program the ESP32 to turn relays on/off to control lights, fans, etc.

- **Process Voice Commands:** You can read microphone inputs and process commands in real time.
- **Optimize Performance:** Your ESP32 runs faster and uses less memory, which is critical for IOT devices.
- **Interface with Hardware:** Embedded C allows your software to directly interact with sensors, relays, and other peripherals.

7.3 Voice Recognition Module / Application



In your project **“Smart Home Assistant using ESP32 and Voice Commands,”** a **Voice Recognition Module** is a key component that allows the system to understand and respond to spoken commands. Here’s a detailed explanation:

1. What is a Voice Recognition Module?

A **Voice Recognition Module** is an electronic device or chip that can:

- **Listen** to spoken words or commands
- **Process** them to identify specific keywords or phrases
- **Send a signal** to a microcontroller (like ESP32) to perform an action

It acts as the “ears and brain” for your smart home system when it comes to understanding voice commands.

2. Types of Voice Recognition Modules

a) Offline/Standalone Modules:

- Examples: **Elechouse Voice Recognition Module V3**
- Can recognize a limited number of pre-trained commands (like “ON,” “OFF,” “Light”)
- Works **without Wi-Fi or cloud services**
- Simple to connect with ESP32 via UART (TX/RX pins)

b) Cloud-Based Voice Recognition:

- Uses services like **Google Assistant, Alexa, or Amazon Voice Service**
- Commands are processed in the cloud, then sent to ESP32 via **Wi-Fi or MQTT**
- Can recognize **unlimited commands**, including natural speech

3. Importance in Your Smart Home Project

For your **ESP32 Smart Home Assistant**, the Voice Recognition Module:

1. **Enables Hands-Free Control**
 - Users can turn lights, fans, or appliances on/off using spoken commands.
 -
2. **Reduces Complexity**
 - The module processes voice, so the ESP32 only needs to act on recognized commands (turn relay ON/OFF).
 -
3. **Enhances User Experience**
 - Makes your smart home system more interactive and “intelligent.”
 -
4. **Flexible Integration**
 - Can work with **relays, LEDs, motors, and IoT platforms** via ESP32.

4. Example in Your Project

Scenario:

- User says: “Turn on the living room light”
- **Voice Recognition Module** detects the command → Sends signal to **ESP32** → ESP32 triggers the **relay** → Light turns on

7.4 IOT PLATFORM



1. Definition of IOT Platform

An **IoT platform** is a **software framework** that allows **Internet of Things (IoT)** devices to:

- Connect to the internet
- Collect and send data
- Receive commands or updates
- Communicate with other devices and applications

In simple terms, it's the "brain and network" that links your smart devices (like ESP32, relays, sensors) to the internet, apps, or cloud services.

2. Key Features of an IOT Platform

1. **Device Management** – Add, remove, and monitor multiple devices remotely.
2. **Data Collection & Analysis** – Store sensor data and track usage (like light on/off history).
3. **Remote Control** – Send commands from a smartphone or web app to ESP32 devices.
4. **Integration** – Connects with voice assistants like **Google Assistant** or **Amazon Alexa**.
5. **Security** – Ensures safe communication between devices and cloud.

3. Importance in Your Smart Home Assistant Project

For your **ESP32 Smart Home Assistant**, an IoT platform:

- Lets your ESP32 receive voice commands from cloud services like Google Assistant.
- Enables you to **control lights, fans, or appliances remotely** via an app or website.
- Helps **monitor the status of your devices** in real-time.
- Supports automation, e.g., **turn lights on at sunset automatically**.

7.5 Mobile Application Integration (Arduino Bluetooth Control)



In our project, we used the **Arduino Bluetooth Controller** mobile application to enable wireless control of home appliances through the **ESP32 microcontroller**.

Connectivity Setup

- The ESP32 has built-in Bluetooth, which allows it to pair directly with the smartphone.
- The mobile app connects via Bluetooth using a unique device name (e.g., ESP-32 Home)
- Once paired, the app establishes a serial communication channel between the phone and ESP32.

Voice Command Integration

- The app supports **voice input**, allowing users to give commands such as:
- “Turn on light”
- “Turn off fan”
- These voice commands are converted into text and then mapped to specific predefined signals (e.g., ‘A’ for light ON, ‘B’ for light OFF).

Data Transmission

- The app sends encoded commands via Bluetooth.
- ESP32 receives the data through its Bluetooth serial interface.
- Based on the received command, ESP32 processes and triggers corresponding GPIO pins.

Device Control Mechanism

- Appliances (like lights, fans) are connected to ESP32 through relay modules.
- When a command is received:
- ESP32 decodes it
- Activates/deactivates relays
- Controls the appliance accordingly

User Interface Features

- Simple button controls for manual operation
- Voice command feature for hands-free control
- Real-time connection status display
- Customizable command buttons

Advantages of Integration

- Wireless and easy to use
- No need for internet (works via Bluetooth)

CHAPTER - 8

CHAPTER - 8: IMPLEMENTATION

The Smart Home Assistant system is designed to automate and control household appliances using voice commands through a smartphone. The core component of this system is the **ESP32**, which enables wireless communication and efficient control of devices.

The system works by integrating a mobile application, such as the **Arduino Bluetooth Controller**, with the ESP32 via Bluetooth. The mobile application provides a user-friendly interface that allows users to control appliances either manually through buttons or by using voice commands.

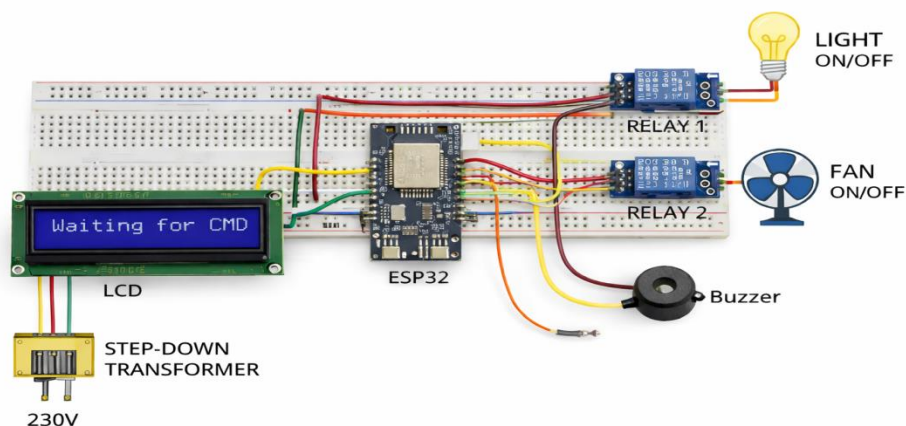
When a user gives a voice command (for example, “Turn on the light”), the mobile application converts the speech into text using built-in voice recognition. This text is then mapped into a predefined command signal and transmitted to the ESP32 through Bluetooth communication. The ESP32 receives this data and processes it using embedded programming.

Based on the received command, the ESP32 activates or deactivates the corresponding GPIO pins connected to a relay module. The relay module acts as an electronic switch, allowing the system to safely control high-voltage appliances such as lights, fans, or other home devices.

This implementation provides a low-cost and efficient solution for home automation without requiring internet connectivity, as it operates entirely on Bluetooth communication. It enhances user convenience by enabling hands-free control and improves accessibility for elderly or physically challenged individuals.

Overall, the Smart Home Assistant demonstrates an effective integration of embedded systems, wireless communication, and voice recognition technology to create a simple yet powerful home automation system.

8.1 Circuit Diagram of Smart Home Assistant Using ESP-32



Circuit Diagram Description of Smart Home Assistant using ESP32 and Voice Commands

The circuit for the smart home assistant system is built around the **ESP32**, which serves as the main controller. It interfaces with a relay module to

control electrical appliances and communicates with a mobile application via Bluetooth.

Main Components in Circuit

- ESP32 Microcontroller
- Relay Module (2-channel or 4-channel)
- Household Appliances (Light, Fan, etc.)
- Power Supply (5V)
- Connecting Wires

ESP32 and Relay Module Connections

- The GPIO pins of ESP32 are connected to the input pins of the relay module:
- GPIO 23 → Relay IN1 (Light)
- GPIO 22 → Relay IN2 (Fan)
- VCC and GND Connections:
- Relay VCC → 5V supply
- Relay GND → ESP32 GND
- ESP32 powered via USB or external supply

Relay to Appliance Connections

- Each relay has three terminals:
 - COM (Common)
 - NO (Normally Open)
 - NC (Normally Closed)
- Appliances are connected using:
 - Live wire → COM
 - NO → Appliance input

When the relay is activated, the circuit closes and the appliance turns ON.

Bluetooth Communication (No Extra Hardware Needed)

- The ESP32 has **built-in Bluetooth**, so no external module (like HC-05) is required.
- It wirelessly receives commands from the mobile app such as the **Arduino Bluetooth Controller**.

Working of the Circuit

1. Power is supplied to ESP32 and relay module
2. Mobile app connects to ESP32 via Bluetooth
3. User gives a voice command
4. Command is sent as a signal to ESP32
5. ESP32 processes the signal and activates the corresponding GPIO pin
6. Relay switches ON/OFF the appliance

Safety Considerations

- Use proper insulation while handling AC connections
- Ensure relay module supports required voltage/current
- Avoid direct contact with live wires
- Use a regulated power supply

8.2 HARDWARE SETUP

Hardware Setup of Smart Home Assistant using ESP32 and Voice Commands

The hardware setup of the smart home assistant system involves interfacing the **ESP32** with a relay module and household appliances to enable wireless control through voice commands.

Components Required

- ESP32 Microcontroller
- Relay Module (2/4 channel)
- Connecting Wires (Jumper wires)
- Breadboard (optional for testing)

- Power Supply (5V)
- Electrical Appliances (Light, Fan, etc.)

Step-by-Step Hardware Setup

1. ESP-32 POWER SUPPLY

- Power the ESP32 using a USB cable or 5V external adapter
- Ensure stable voltage for proper operation

2. CONNECTING RELAY MODULE TO ESP-32

- Connect relay **VCC** → 5V
- Connect relay **GND** → ESP32 GND
- Connect control pins:
 - GPIO 23 → IN1 (Light)
 - GPIO 22 → IN2 (Fan)

These GPIO pins will control the switching of appliances.

3. CONNECTING APPLIANCES TO RELAY MODULE

- Each relay channel has:
 - COM (Common)
 - NO (Normally Open)
 - NC (Normally Closed)

Connection:

- Live wire → COM
- NO → Appliance
- Neutral wire → Direct to appliance

When relay is triggered, the appliance turns ON.

4. BLUETOOTH SETUP

- No external module is required since ESP32 has built-in Bluetooth
- Pair ESP32 with mobile using apps like **Arduino Bluetooth Controller**

5. FINAL ASSEMBLY

- Place components on a breadboard or enclosure
- Ensure all connections are tight and insulated
- Keep high-voltage and low-voltage sections separated

Working of Hardware Setup

- ESP32 receives commands via Bluetooth
- Processes voice command signals
- Sends output to relay module
- Relay switches appliances ON/OFF

Conclusion

The hardware setup forms the backbone of the system, enabling the ESP32 to interface safely with real-world appliances and execute voice-based control efficiently.

8.3 Software Development

The software development of this system focuses on programming the **ESP32** and configuring the mobile application to enable communication and control of home appliances through voice commands.



Development Tools Used

- **Arduino IDE**
Used to write, compile, and upload code to the ESP32.
- **Arduino Bluetooth Controller**
Used for sending voice commands and control signals via Bluetooth.



Programming the ESP32

- The ESP32 is programmed using Embedded C in Arduino IDE.
- Required libraries:
 - Bluetooth Serial library (for Bluetooth communication)

Main functions in the code:

- Initialize Bluetooth communication
- Read incoming data from mobile app
- Compare received commands
- Control GPIO pins accordingly

Software Workflow

1. Initialize Bluetooth on ESP32
2. Wait for connection from mobile app
3. Receive command (character or string)
4. Process command using conditional statements
5. Trigger corresponding GPIO output
6. Repeat continuously

Voice Command Processing

- The mobile app captures voice input
- Converts speech to text
- Maps text to predefined commands:
 - “Light ON” → ‘A’
 - “Light OFF” → ‘B’
 - “Fan ON” → ‘C’
 - “Fan OFF” → ‘D’
- These commands are sent via Bluetooth to ESP32

Mobile Application Configuration

- Install the Arduino Bluetooth Controller app
- Pair with ESP32 Bluetooth device
- Configure buttons/voice commands with specific characters
- Test commands for proper response

Features of Software

- Real-time response to commands
- Simple and efficient logic
- Supports both voice and manual input
- Easy to modify and expand

8.4 Integration of Voice Commands

Integration of Voice Commands

The integration of voice commands in the smart home assistant system allows users to control appliances using spoken instructions. This feature is implemented using a mobile application and the **ESP32** with Bluetooth communication.



A mobile application such as the **Arduino Bluetooth Controller** is used to capture voice input from the user. The app utilizes the smartphone's built-in speech recognition system to convert spoken commands into text.

After conversion, the text is mapped to predefined command codes. For example:

- “Light ON” → ‘A’
- “Light OFF” → ‘B’
- “Fan ON” → ‘C’
- “Fan OFF” → ‘D’

These command codes are then transmitted to the ESP32 via Bluetooth.

The ESP32 continuously receives and processes the incoming data. Based on the received command, it controls the corresponding GPIO pins connected to the relay module. The relay then switches the appliances ON or OFF accordingly.

This integration enables hands-free operation, improves user convenience, and makes the system more efficient and accessible. However, the performance depends on the accuracy of voice recognition and may be affected by noise or unclear speech.

The integration of voice commands in the smart home assistant system enables users to control appliances through spoken instructions, making the system more user-friendly and accessible. This functionality is achieved by combining a mobile application with the **ESP32** via Bluetooth communication.

The system uses a mobile application such as the **Arduino Bluetooth Controller**, which includes built-in voice recognition capabilities. When a user speaks a command, the application captures the audio input and converts it into text using the smartphone's speech recognition engine.

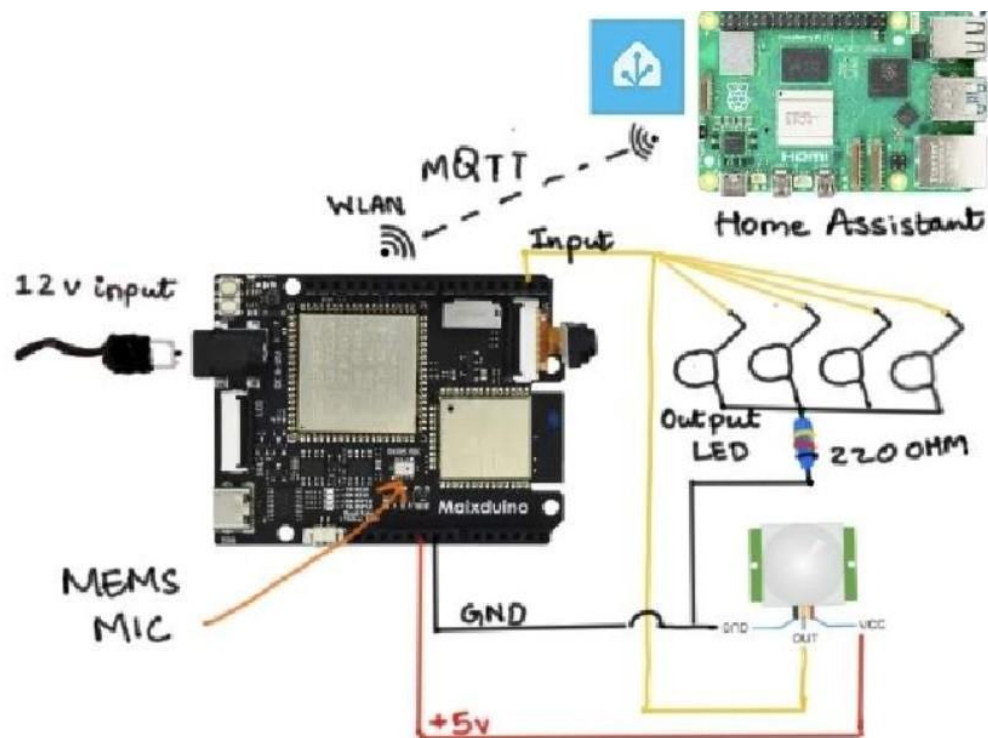
Once the voice input is converted into text, the application processes it and maps it to predefined control commands. For example, voice instructions like “Turn on the light” or “Turn off the fan” are translated into specific characters or signals (such as ‘A’, ‘B’, ‘C’, ‘D’). These signals are then transmitted wirelessly to the ESP32 through Bluetooth.

On the receiving end, the ESP32 continuously monitors incoming Bluetooth data. When a command is received, it is compared with predefined conditions in the program. Based on the matched command, the ESP32 activates or deactivates the corresponding GPIO pins connected to the relay module. This action results in switching the connected appliances ON or OFF.

This integration eliminates the need for physical interaction with switches and enhances convenience by enabling hands-free operation. It is especially beneficial for elderly or differently-abled individuals, as it simplifies the process of controlling home devices.

However, the system relies on the accuracy of voice recognition provided by the mobile device, and performance may vary depending on background noise and pronunciation. Despite these limitations, voice command integration significantly improves the usability and modern functionality of the smart home system.

8.5 TESTING PROCEDURE



Testing Procedure of Smart Home Assistant using ESP32 and Voice Commands

The testing procedure is carried out to ensure that all components of the system work correctly and efficiently. It verifies the communication between the mobile application and the **ESP32**, as well as the proper functioning of connected appliances.

Step-by-Step Testing Procedure

1. Hardware Verification

- Check all circuit connections (ESP32, relay module, appliances)
- Ensure proper power supply is provided
- Verify that there are no loose or incorrect connections

2. Power ON Test

- Turn ON the power supply to ESP32
- Confirm that the ESP32 powers up (LED indication)
- Ensure relay module receives power

3. Bluetooth Connectivity Test

- Open the **Arduino Bluetooth Controller**
- Pair the smartphone with ESP32 Bluetooth (e.g., *ESP32_Home*)
- Check if the connection is successfully established

4. Command Transmission Test

- Send manual commands using app buttons
- Verify whether ESP32 receives the correct data
- Observe serial monitor (optional) for received commands

5. Voice Command Test

- Use voice input feature in the app
- Speak commands like:
 - “Turn ON light”
 - “Turn OFF fan”
- Check if commands are correctly recognized and transmitted

6. Relay Operation Test

- Observe relay switching when commands are sent
- Listen for clicking sound of relay activation
- Ensure correct relay responds to each command

7. Appliance Control Test

- Check if appliances (light, fan) turn ON/OFF correctly
- Verify response time and accuracy
- Repeat multiple times for consistency

8. Range Testing

- Move the smartphone away from ESP32
- Test system operation at different distances
- Note the maximum effective Bluetooth range

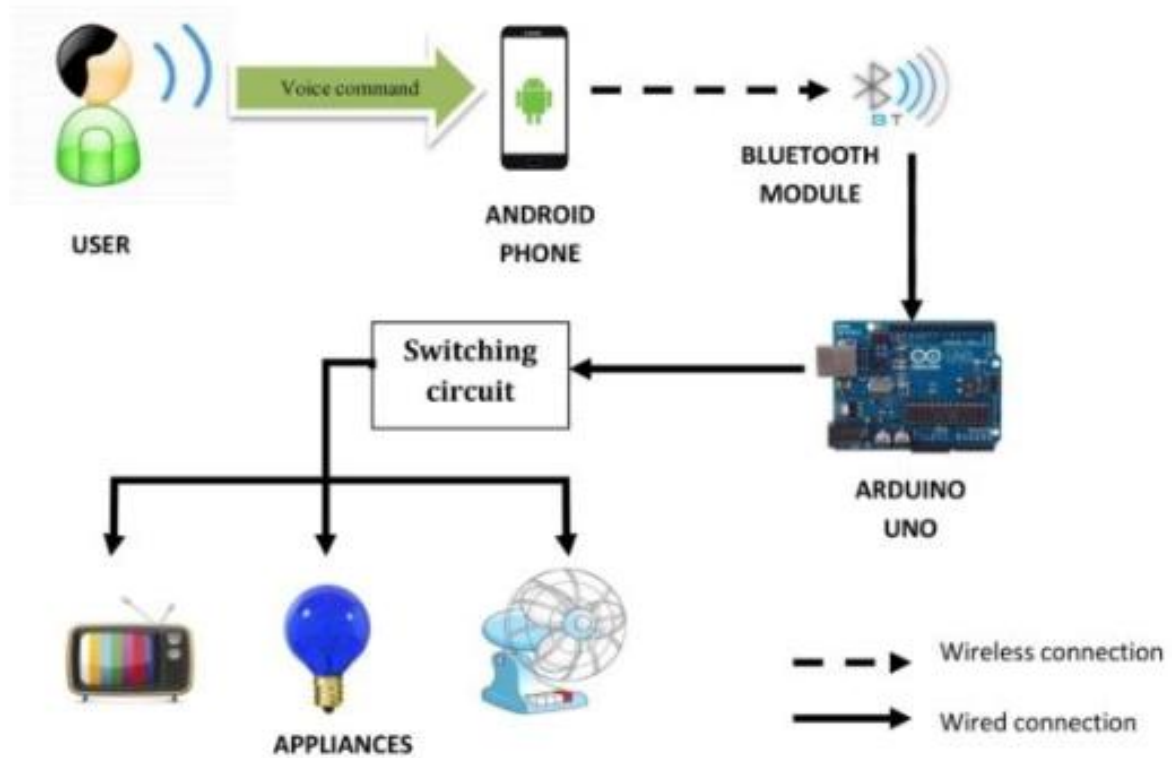
9. Error Handling Test

- Send invalid or random commands
- Check system stability and response

CHAPTER - 9

CHAPTER - 9: WORKING OF THE SYSTEM

9.1 VOICE COMMAND PROCESSING



Description of Voice Command Processing

Voice command processing is a key feature of the smart home assistant system, enabling users to control appliances commands and executing through spoken instructions. This process involves converting human speech into machine-understandable them using the **ESP32**.

Initially, the user gives a voice command through a mobile application such as the **Arduino Bluetooth Controller**. The application uses the smartphone's built-in speech recognition engine to capture and convert the spoken words into text.

Once the speech is converted into text, the application processes the input and matches it with predefined command patterns. These commands are then encoded into specific characters or signals. For example:

- “Turn on light” → ‘A’
- “Turn off light” → ‘B’
- “Turn on fan” → ‘C’
- “Turn off fan” → ‘D’

- “Turn on ALL loads”→’E’
- “Turn off ALL loads”→’F’

These encoded signals are transmitted wirelessly to the ESP32 via Bluetooth communication.

On the ESP32 side, the system continuously listens for incoming data. When a command is received, it is compared with predefined conditions in the program. Based on the matched condition, the ESP32 generates appropriate output signals through its GPIO pins.

These output signals are sent to the relay module, which acts as a switch to control electrical appliances. As a result, the respective device is turned ON or OFF according to the user’s voice command.

This entire process—from speech input to appliance control—occurs in real time, providing a seamless and hands-free user experience. However, the accuracy of the system depends on the quality of voice recognition, pronunciation, and surrounding noise conditions.

9.2 COMMUNICATION VIA BLUETOOTH

ESP32 Bluetooth Data Communication

The **ESP32** has **in-built Bluetooth Classic and BLE (Bluetooth Low Energy)** support. It can send and receive data wirelessly without using external modules like HC-05.

There are **two types of Bluetooth in ESP32**:

1. **Bluetooth Classic (SPP – Serial Port Profile)**
2. **Bluetooth Low Energy (BLE)**



ESP32 with Bluetooth and Bluetooth Low Energy

ESP32 boards are great for IoT projects because they support Wi-Fi, Bluetooth Classic, and Bluetooth Low Energy. This is a comprehensive guide that compiles several of our previous projects in a logical way to make the learning process easier.

What is Bluetooth?



Bluetooth is a wireless technology that enables devices to communicate over short distances.

It's commonly used for connecting devices like headphones to phones or linking a keyboard or mouse to a computer. The ESP32 board also supports Bluetooth along with Wi-Fi, making it an excellent choice for IOT projects. This capability allows ESP32-based projects to wirelessly exchange data or connect to other Bluetooth-enabled devices seamlessly.

This variant of Bluetooth is also referred to as “Bluetooth Classic” or simply “Bluetooth”.

What is Bluetooth Low Energy?



Bluetooth Low Energy, BLE for short (also called Bluetooth Smart), is a power-conserving variant of Bluetooth. BLE's primary application is short-distance transmission of small amounts of data (low bandwidth). Unlike Bluetooth which is always on, BLE remains in sleep mode constantly except for when a connection is initiated.

Bluetooth Classic vs Bluetooth Low Energy

So, what are the main differences between Bluetooth Classic and Bluetooth Low Energy?

Bluetooth Classic is known for higher data transfer rates, making it suitable for applications like audio streaming and file transfer. It consumes more power, making it less ideal for battery-operated devices.

On the other hand, **Bluetooth** is designed for low power consumption, making it perfect for devices like IOT gadgets and wearables, and is also a great solution for the ESP32 in IoT and Home Automation applications. BLE operates with lower data transfer rates but is energy-efficient and works well in short-range scenarios.

Another big difference between the two versions of Bluetooth is the way used to transfer data. Bluetooth Classic uses something similar to Serial Communication, while Bluetooth Low Energy uses a client-server model, where it employs the GATT (Generic Attribute Profile) to structure data.

Parameters	Bluetooth Classic	Bluetooth Low Energy (BLE)
Power Consumption	Higher power consumption	Low power consumption
Data Transfer Rate	Higher data transfer rates	Lower data transfer rates
Range	Longer range	Shorter range
Application Examples	Audio streaming, file transfer	IOT devices, Wearables, Smart home
Data Transfer	Serial Port Profile (SPP)	Generic Attribute Profile (GATT)

Bluetooth Classic with the ESP32

In terms of programming, using Bluetooth Classic is much simpler than setting up the ESP32 as a BLE device. If you've already programmed an Arduino board with a Bluetooth module like the HC-06, it is very similar. It uses the standard serial protocol and functions.



We have a detailed tutorial explaining how to exchange data with the ESP32 via Bluetooth Classic. Check the link below:

[ESP32 Bluetooth Classic with Arduino IDE – Getting Started](#) (only compatible with Android Smartphones)

It also includes a sample project showing how to send sensor readings to your smartphone and control an LED via Bluetooth Classic.

Bluetooth with the ESP32 – Getting Started Guides

Understanding how Bluetooth Low Energy works is a bit more complicated than Bluetooth Classic. But, don't worry, we have several guides focused on different subjects with project examples so that you easily understand how it works.

ESP32 Getting Started with Bluetooth Low Energy

The best way to get started with BLE and the ESP32 is to start by learning about basic concepts like BLE Server and Client, GATT, BLE Profiles, Services, and Characteristics. You can start with the following tutorial:

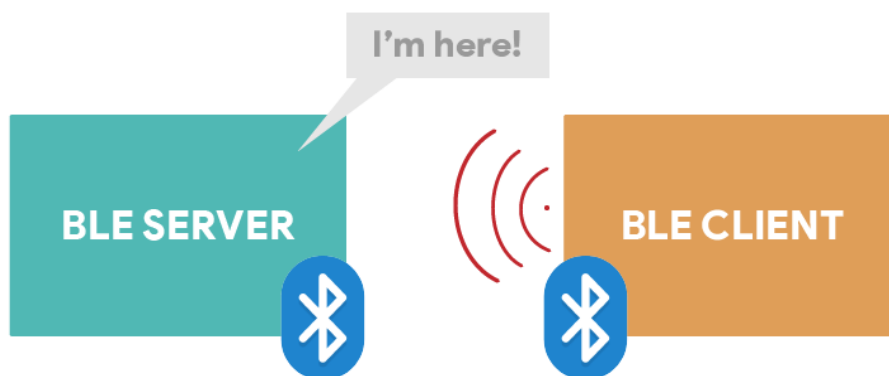
- [Getting Started with ESP32 Bluetooth Low Energy \(BLE\) on Arduino IDE](#)

This tutorial explains BLE's most important theoretical concepts and tests some basic BLE examples on the ESP32 to set it as a BLE Client and as a BLE Server.

ESP32 Bluetooth Low Energy Client and Server

With Bluetooth Low Energy, there are two types of devices: the server and the client. The ESP32 can act either as a client or as a server.

The server advertises its existence, so it can be found by other devices and contains data that the client can read. The client scans the nearby devices, and when it finds the server it is looking for, it establishes a connection and listens for incoming data. This is called point-to-point communication.



Bluetooth Classic (Serial Communication)

This works similar to **UART communication**. Data is sent and received like serial data.

◆ Features

- Easy to use
- Works with Android apps (like Serial Bluetooth Terminal)
- Suitable for robot control, sensor data transfer

How It Works

1. Upload code to ESP32
2. Turn ON Bluetooth in mobile
3. Pair with **ESP32_BT**
4. Open Serial Bluetooth Terminal app
5. Send/Receive data

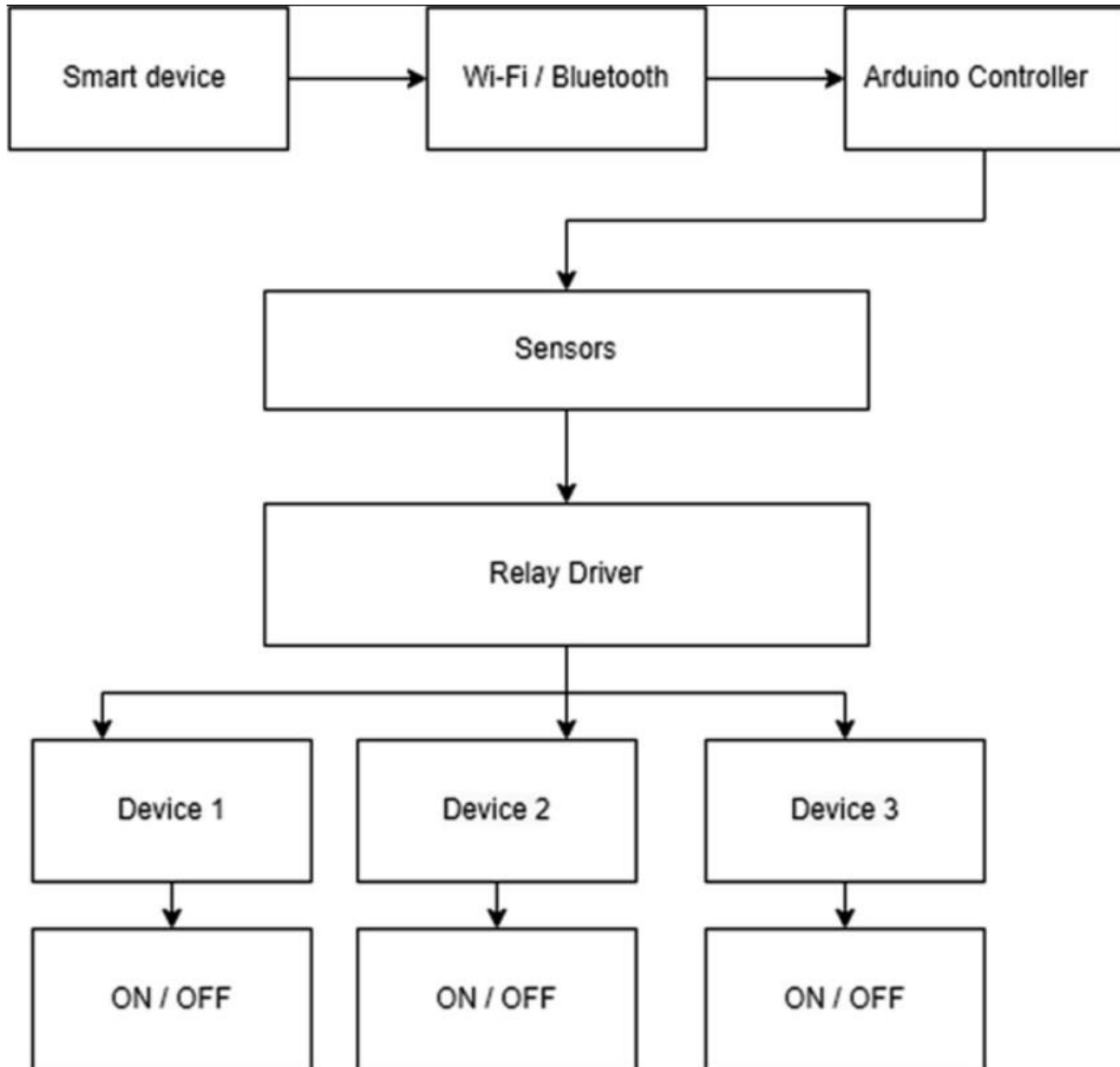
Applications

- Gesture controlled robot
- Home automation
- Sensor monitoring
- Data logging
- Wireless control systems

Advantages of ESP32 Bluetooth

- No external module required
- Dual mode (Classic + BLE)
- Low cost
- High speed communication
- Easy integration with Arduino IDE

9.3 DEVICE CONTROL MECHANISM



Device Control Mechanism

The **Device Control Mechanism** in the Smart Home Assistant system defines how appliances are operated using the **ESP32** and voice or manual commands from the mobile application.

Components Involved

1. **ESP32 Microcontroller** – Processes incoming commands and controls outputs.
2. **Relay Module** – Acts as an electrically controlled switch for high-voltage appliances.
3. **Appliances** – Devices such as lights, fans, and other household electronics.
4. **Mobile App (Arduino Bluetooth Controller)** – Sends commands via Bluetooth.

◆ Working Mechanism

1. Command Input

- The user gives a command either through:
 - **Voice commands** (app converts speech to text)
 - **Manual buttons** on the app interface
 -

2. Command Transmission

- Commands are converted to predefined signals (e.g., 'A', 'B', 'C', 'D')
- Transmitted wirelessly via Bluetooth to ESP32
-

3. ESP32 Processing

- ESP32 continuously monitors its Bluetooth serial input
- Received command is compared against predefined conditions in the program
- ESP32 determines which GPIO pin should be activated
-

4. Relay Switching

- ESP32 sets the GPIO pin HIGH or LOW depending on the command
- The relay module receives the signal and switches the connected appliance ON or OFF
-

5. Appliance Operation

- When the relay is activated, the appliance receives power and turns ON
- When the relay is deactivated, the appliance turns OFF

◆ Flow of Device Control

1. User → Mobile App → Voice/Manual Command
2. Mobile App → Bluetooth → ESP32
3. ESP32 → GPIO → Relay Module
4. Relay → Appliance ON/OFF

✓ Advantages

- Hands-free and convenient operation
- Safe control of high-voltage devices via relays
- Quick and reliable response
- Easily expandable for additional appliances

The Traffic Sign Detection and Recognition system can be further enhanced in several ways to improve its performance, scalability, and real-world applicability. Although the current system uses a LeNet-based CNN for classification, future improvements can include implementing advanced deep learning architectures such as Resnet, VGG, or YOLO for higher accuracy and better real-time object detection capabilities. These models can handle larger datasets and complex traffic scenarios more effectively.

Another important enhancement is expanding the dataset to include a wider variety of traffic signs from different countries and environments. Incorporating diverse lighting conditions, weather variations (rain, fog, night), and occluded signs will improve model robustness and generalization. Data augmentation techniques can also be applied to increase training data diversity.

The system can be upgraded from classification-only to full object detection by integrating bounding box detection methods. This will allow the system to locate traffic signs within larger frames rather than classifying resized full images. Additionally, deploying the model on embedded systems such as Raspberry Pi, NVIDIA Jetson Nano, or mobile devices can make it suitable for real-time vehicle applications.

Integration with Advanced Driver Assistance Systems (ADAS) and autonomous vehicles is another promising direction. The system can be combined with lane detection, pedestrian detection, and collision avoidance systems to create a complete intelligent driving solution.

Cloud integration and IoT connectivity can also be implemented for centralized traffic monitoring and smart city applications. Furthermore, performance optimization techniques such as model compression and pruning can improve speed and reduce hardware requirements.

Overall, the future scope of this project includes improving accuracy, expanding functionality, enhancing real-time capabilities, and enabling deployment in real-world intelligent transportation systems.

CHAPTER - 10

CHAPTER-10: RESULTS AND DISCUSSION

10.1 OUTPUT RESULTS

The experimental results of the Voice Controlled Home Automation System using ESP32 confirm that the system operates reliably and efficiently under normal indoor conditions. When powered ON, the ESP32 successfully initializes Bluetooth communication and displays system readiness on the I²C LCD. The Android voice control application establishes a stable Bluetooth connection with minimal pairing delay.



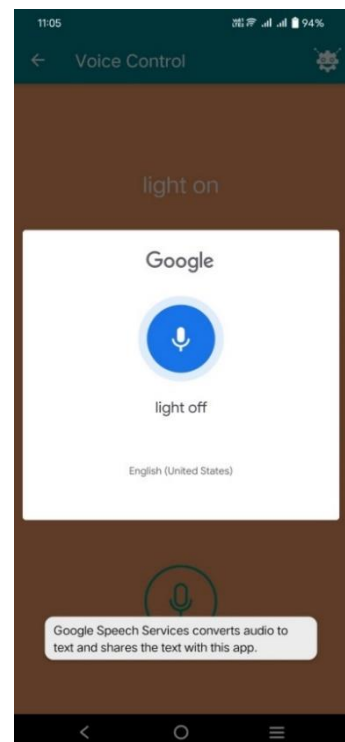
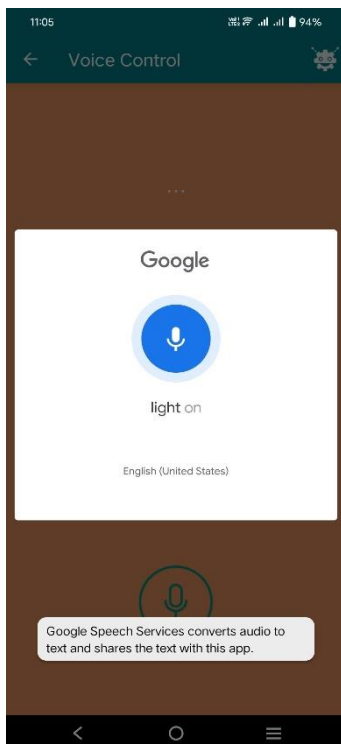
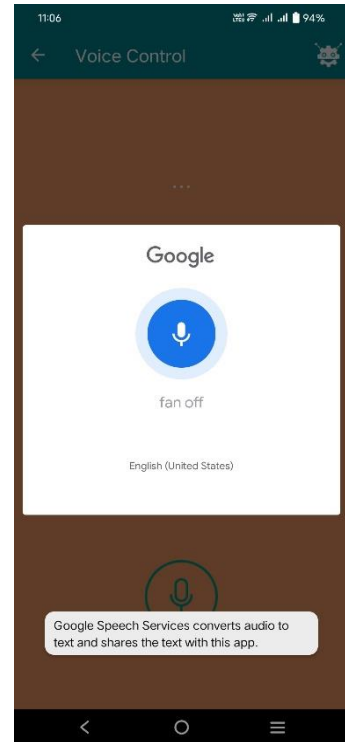
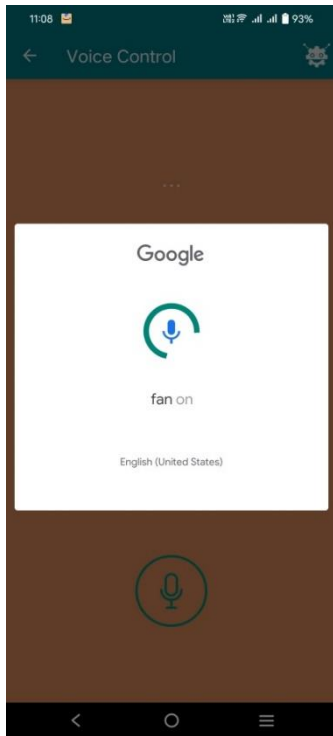
Fig:1



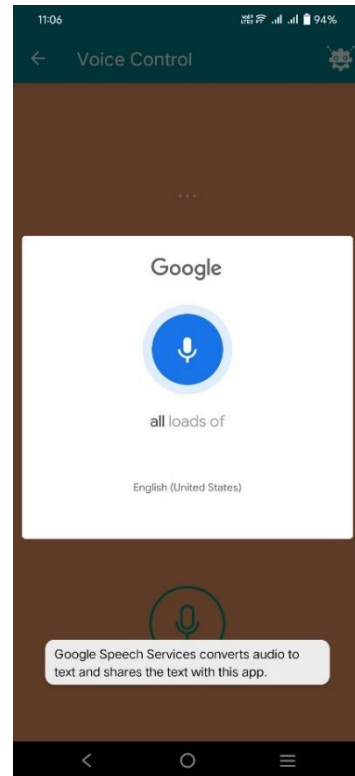
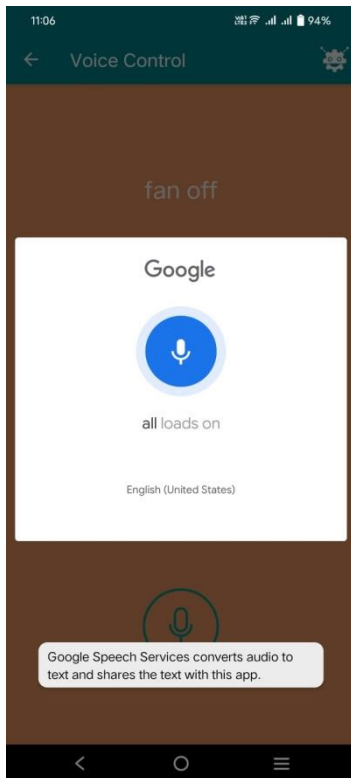
Fig:2

In fig1 it is initial step of the project, In fig2 we setup the commands in settings.

These commands are transmitted via Bluetooth and correctly interpreted by the ESP32. The response time between issuing a voice command and load activation is observed to be less than one

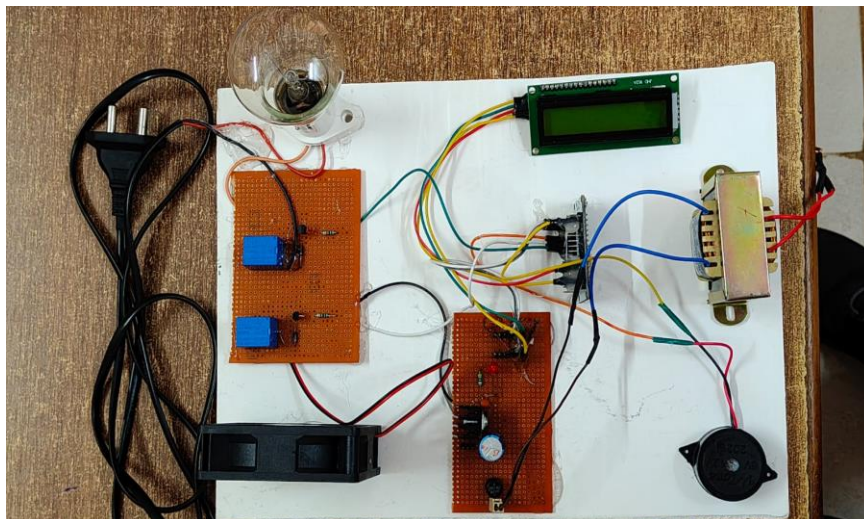


The light and fan response correctly to relay control commands without electrical noise or false triggering. The DC fan and DC motor also function as expected, demonstrating.

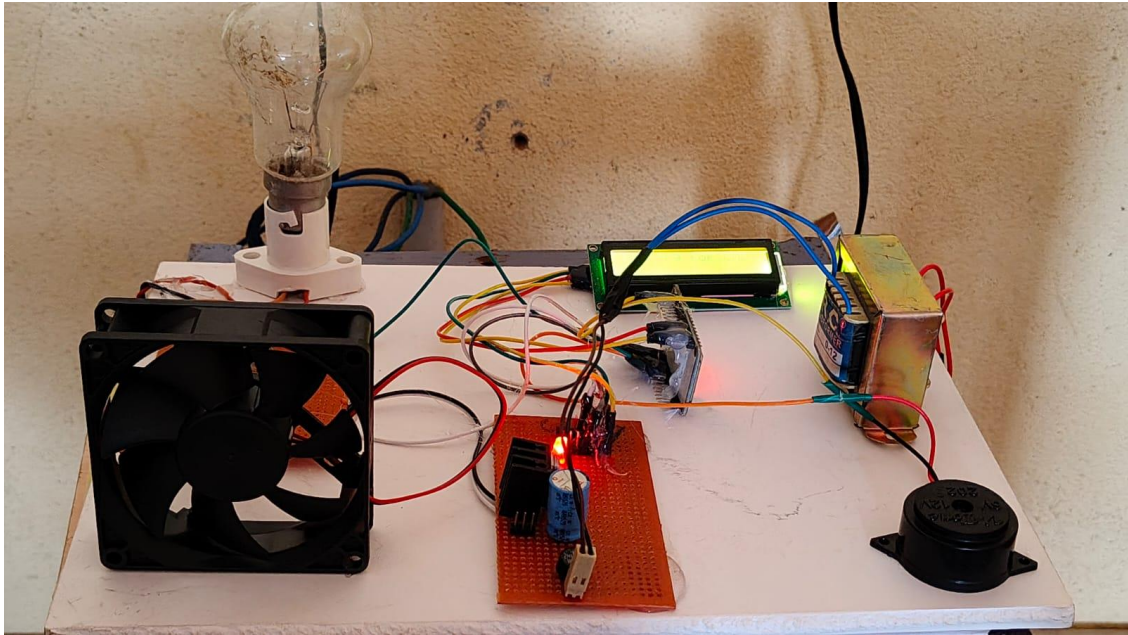


The Android voice control application establishes a stable Bluetooth connection with minimal pairing delay.

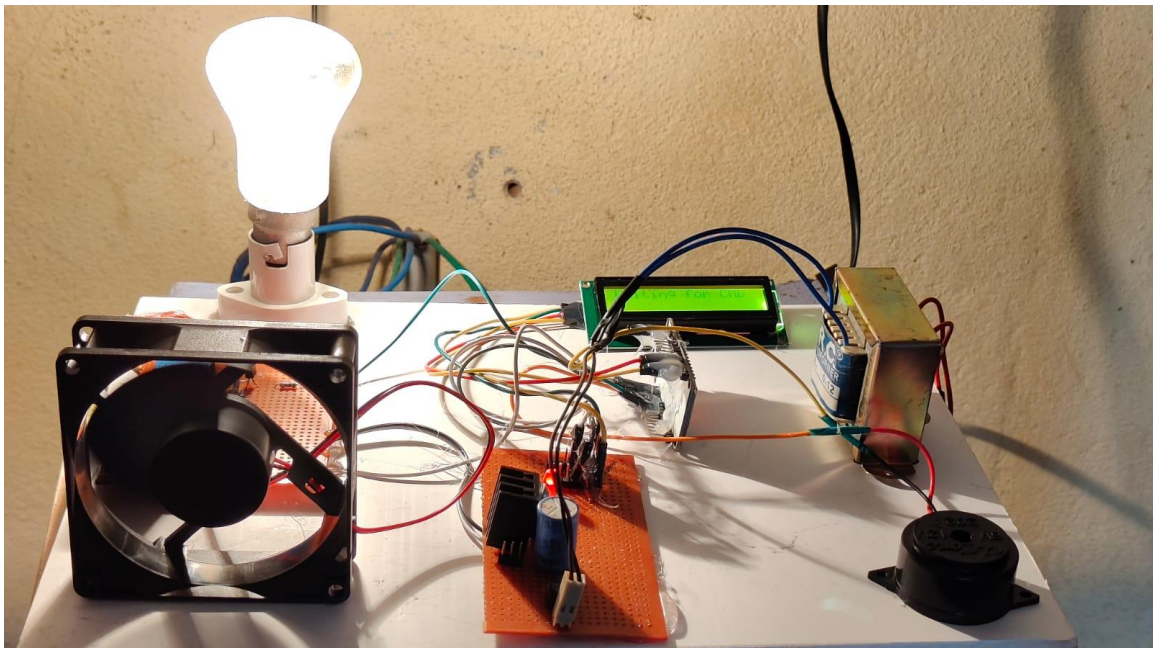
The Voice commands issued through the Android application are accurately converted into text using the smartphones speech recognition engine. These commands are transmitted via Bluetooth and correctly interpreted by the ESP32. The response time between issuing a voice command and load activation is observed to be less than one second, indicating fast and real-time operation.



In above figure these is the hardware development kit



It represents a system used for automating home appliances, such as controlling devices, alerts, and environmental monitoring.



The system works by using a ESP-32 to receive input through a Bluetooth and process commands.

Based on the input, it activates relays to turn ON/OFF devices like the bulb, fan, and buzzer, while displaying status on the LCD.

CHAPTER-11

CHAPTER – 11: CONCLUSION AND FUTURE SCOPE

11.1 CONCLUSION

The Smart Home Assistant Robot using ESP32 and Voice Commands presents a modern, efficient, and user-friendly approach to home automation. By integrating the powerful ESP32 microcontroller with voice recognition technology, the system enables seamless, hands-free control of household appliances and robotic functions.

Its ability to respond to voice commands, perform real-time actions, and provide feedback enhances convenience, accessibility, and overall user experience. The inclusion of IoT connectivity further allows remote monitoring and control, making the system flexible and scalable for future advancements.

In this commands with features like obstacle detection, appliance automation, and interactive communication, the proposed model proves to be reliable and practical for smart living environments. Additionally, its cost-effectiveness and adaptability make it suitable for a wide range of applications, including assistance for elderly and physically challenged individuals. Overall, this project demonstrates how embedded systems and voice-based technologies can be combined to create an intelligent, responsive, and efficient smart home solution, contributing to the evolution of next-generation automation systems.

The development of a Smart Home Assistant Robot using ESP32 and voice commands highlights the growing importance of automation and intelligent systems in daily life. This project successfully combines embedded systems, wireless communication, and voice recognition to create a smart and interactive environment. The ESP32 serves as a powerful and efficient controller, enabling smooth communication between hardware components and user commands. The system reduces manual effort by allowing users to control appliances and receive information through simple voice instructions.

11.2 FUTURE SCOPE

The future scope of the Smart Home Assistant Robot using ESP32 and voice commands is vast and promising, with significant opportunities for enhancement and innovation. In the coming years, the system can be improved by integrating advanced artificial intelligence and natural language processing to enable more accurate and human-like interactions. Support for multiple languages and personalized user experiences can make the system more accessible to a wider audience.

The addition of face recognition and biometric authentication can enhance security, while integration with smart surveillance systems and real-time video monitoring achieve greater compatibility and functionality within existing.

The smart home ecosystems can provide better home protection. By connecting the robot with popular platforms like Google Assistant and Amazon Alexa, the system can. furthermore, the use of machine learning can allow the robot to learn user habits and automate tasks intelligently, such as controlling lighting, temperature, and appliances based on daily routines. Future developments may include improved navigation using advanced technologies like LiDAR and SLAM for accurate indoor mapping and obstacle avoidance.

The Integration with healthcare monitoring systems can make the robot highly beneficial for elderly and physically challenged individuals by providing emergency alerts, fall detection, and health tracking features. The incorporation of IoT platforms such as Blynk and Thing speak can enable efficient remote monitoring, data analysis, and system control through mobile or web applications.

In Additionally, advancements in connectivity such as 5G and edge computing can reduce latency and improve real-time performance. The system can also evolve to include renewable energy solutions, such as solar power, for sustainable operation. Future versions may support integration with smart city infrastructure, industrial automation, and advanced robotics features like robotic arms for performing physical tasks.

To the performance with continuous improvements in security, scalability, and user interaction, the Smart Home Assistant Robot has the potential to transform into a fully autonomous, intelligent system that plays a crucial role in modern smart homes and digital living environments.

11.3 REFERENCES

1. Espressif Systems, “**ESP32 Series Datasheet**”, Version 4.9, Espressif Systems, 2023.
2. Espressif Systems, “**ESP32 Technical Reference Manual**”, Espressif Systems, 2023
3. Muhammad Ali Mazidi, Sarmad Naimi, and Sepehr Naimi, The AVR Microcontroller and Embedded Systems: Using Assembly and C, Pearson Education, 2018.
4. Simon Monk, Programming Arduino: Getting Started with Sketches, McGraw-Hill Education, 2016
5. Android Developers, “**Speech Recognition API Documentation**”, Google Developers, 2023.
6. Cireşan, D. Meier, U. Masci, J. & Schmidhuber, J. (2012). Multi-Column Deep Neural Network for Traffic Sign Classification. Neural Networks, 32, 333–338. (High-accuracy deep learning model for traffic sign recognition.)
7. NXP Semiconductors, “**I²C-bus Specification and User Manual**”, Rev. 7.0, 2021.

8. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). (Real-time object detection framework relevant for future enhancements.)
9. Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). ImageNet Classification with Deep Convolutional Neural Networks. (Landmark paper demonstrating CNN effectiveness in image classification.)
10. TensorFlow Documentation. (n.d.). TensorFlow Official Documentation. Retrieved from <https://www.tensorflow.org>
10. Keras Documentation. (n.d.). Keras: Deep Learning for Humans. Retrieved from <https://keras.io>
11. OpenCV Documentation. (n.d.). Open source Computer Vision Library. Retrieved from <https://opencv.org>
12. Python Software Foundation. (n.d.). Python Programming Language Documentation. Retrieved from <https://www.python.org>



NAME : INJARAPU SIVA KRISHNA
ROLL NO : 22JN1A04D3
PH NO : 9390245314
EMAIL ID : injarapusivakrishna@gmail.com



NAME : SHAIK ABRARUL HUK
ROLL NO : 22JN1A04E0
PH NO : 8328515923
EMAIL ID : abrarulhukshaik2003@gmail.com



NAME : SHAIK MOHAMMED THAHIR
ROLL NO : 22JN1A04E6
PH NO : 9014202851
EMAIL ID : Shaikmohammedthahir26@gmail.com



NAME : SRIPATHI VAMSI KISHORE
ROLL NO : 22JN1A04F8
PH NO : 9908480801
EMAIL ID : vamsivamsikishore740@gmail.com